

课程笔记

生产级 AI 作战手册：企业规模智能体部署的 五支柱框架

sss & Codex

2026 年 6 月 28 日



视频作者/频道: AI Engineer

发布日期: 2026 年 6 月 18 日

视频时长: 37 分 05 秒

视频链接: <https://www.youtube.com/watch?v=ObTPqBGsEbA>

目录

来源说明	3
1 从 Demo 到 Production：为什么模型优先路线会失败	4
1.1 讲者与场景：Databricks 的生产化视角	4
1.2 熟悉的失败路径：选模型、做 Demo、上线后失控	4
1.3 三个生产化缺口：看不见、量不准、管不住	5
1.4 五支柱框架总览：Evaluation、Observability、Data Foundation、Orchestration、Governance	6
1.5 本章小结	7
2 Evaluation First：把评估当成 AI 系统规格说明	8
2.1 先定义成功：业务指标要有数字	8
2.2 真实案例集：从领域专家和一线坐席中收集 Golden Dataset	9
2.3 自动化评估流水线：从离线测试走向线上持续测量	9
2.4 三层评估：确定性、语义性、行为性	10
2.5 本章小结	11
3 Observability：让每一次 AI 决策可回放	11
3.1 从一次 overdraft fee 争议看 trace 的必要性	11
3.2 失败会话要能在五分钟内回放	12
3.3 线上监控与 fallback：重试、升级、转人工	13
3.4 本章小结	13
4 Data Foundation：Question Data 与 Tracking Data 的双重地基	14
4.1 为什么数据地基最耗时：人会容错，智能体会自信地答错	14
4.2 Question Data：回答问题所需的数据	14
4.3 Tracking Data：观测、审计、评估所需的轨迹数据	15
4.4 Databricks 数据智能平台示例：Cloud Storage、Delta Lake、Unity Catalog、Mosaic AI、Genie	15
4.5 跨框架集中式可观测性：从 CrewAI、LangChain、云平台到统一表	16
4.6 本章小结	17
5 Orchestration and Governance：多智能体协作与生产控制面	18
5.1 一个智能体到多个智能体：复杂度为什么会指数式上升	18
5.2 Orchestrator-Worker：中央控制与可追责日志	19
5.3 Choreography：事件总线与并行自治	20
5.4 Human in the Loop：阈值、置信度与人工接管	20
5.5 AI Governance：审计轨迹、PII 预验证、Prompt 版本化、模型变更管理	21
5.6 Agent Bricks：Databricks 对五支柱的产品化封装	22
5.7 本章小结	23

6 Case Study: 零售银行聊天机器人如何重新排序	23
6.1 原始失败: 六个月 POC、约 85K 成本、生产失败原因不可见	23
6.2 Weeks 1-2: 先建 Evaluation Layer	25
6.3 Weeks 3-6: 重建 Foundation Layer	25
6.4 Weeks 7-8: 最后才选模型并上线	26
6.5 Six weeks post-launch: 指标、CSAT 下滑与 stale policy 修复	27
6.6 本章小结	28
7 Production Incident Playbook: AI 生产事故如何闭环	28
7.1 多数团队有部署计划, 事故计划经常缺席	28
7.2 Detect: 用 Eval Dashboard 先发现指标下滑	29
7.3 Diagnose: 用 Distributed Tracing 找到 correct 到 wrong 的转折点	30
7.4 Contain: Prompt Versioning、转人工、降低爆炸半径	30
7.5 Fix and Prevent: Test Case Library 与 Eval Suite Expansion	30
7.6 明天能做什么: 定义成功、收集好答案、写一个最小评估流水线	31
7.7 本章小结	32
8 总结与延伸: 三个容易漏掉的工程教训	32
8.1 Lesson 1: Test Case Library 需要 owner 和分类治理	32
8.2 Lesson 2: Prompt Versioning 的 commit message 必须能解释原因	33
8.3 Lesson 3: Layer 3 behavioral evals 会很贵, 需要 CI 策略	33
8.4 资源与结束语: 模板、Tracing 指南、LinkedIn/newsletter	34
8.5 全篇综合: 从五支柱压缩成一个生产闭环	34
8.6 本章小结	35

来源说明

本笔记的主要字幕证据来自 YouTube 自动英文字幕 (**en-orig**)，该视频本地元数据未发现人工字幕轨。关键数字、产品名和图示结论优先用视频画面核对；仅来自口述字幕的内容已尽量采用概括性转述。读者在复核专有名词、工具名或细粒度措辞时，应同时查看原视频画面与字幕，因为 ASR 可能把发音相近的术语混淆。

1 从 Demo 到 Production: 为什么模型优先路线会失败

1.1 讲者与场景: Databricks 的生产化视角

这一讲的主题是生产级 AI (Production AI): AI 系统已经进入真实用户、真实数据、真实责任链和真实监管压力之中。讲者 Sandipan Bhaumik, 也在开场自称 Sandy, 是 Databricks 的 data and AI technical lead; 在加入 Databricks 之前, 他曾在 Amazon Web Services 担任 data and AI 方向的 principal architect。这个背景很关键, 因为本讲的视角超出单次模型演示, 来自企业平台、分布式系统、数据治理和客户上线经验的交叉地带。



图 1: 官方封面图保留了 Production AI Playbook 的核心定位: 生产化问题高于单点模型能力。

讲者强调, 过去两年客户都在快速试验新 AI 技术。这里的“新”指组织突然开始把大语言模型、检索、工具调用和智能体能力接入实际业务流程。B2B 软件公司和金融服务等受监管行业给了他同一个教训: 演示原型 (Demo) 可以在有限环境里显得顺滑, 生产环境会迅速暴露测量、追踪、数据、协作和治理问题。

“生产级”为什么高于更漂亮的 Demo

Demo 像样板间: 数据固定、场景受控、观众知道看什么。Production AI 像 24 小时运行的调度中心: 用户问题不可预测, 后端数据会变化, 系统需要记录每一步, 出错后还要说明责任和补救路径。二者的差距主要来自运行制度、责任链和反馈机制。

因此, 本章先建立整场讲座的基本判断: 企业 AI 上线失败的常见根因, 比“选错了一个模型”这种解释更深。很多团队在项目一开始就把模型选择当成主线, 缺少提前定义成功标准、链路追踪、数据地基、编排方式和治理控制面的动作。

1.2 熟悉的失败路径: 选模型、做 Demo、上线后失控

讲者把他在客户会议中反复看到的路径称为“你已经熟悉的模式”。第一步是从模型争论开始: 用 GPT, 还是用 Claude; 选哪家提供商; 哪个榜单分数更高。第二步是围绕模型快速堆功能, 在可

控数据集、少量场景和演示脚本里完成一个效果不错的应用。第三步是向领导演示，获得签字，上线。第四步才是真实用户开始提出难以预测的问题，团队才发现系统没有办法解释自己的行为。

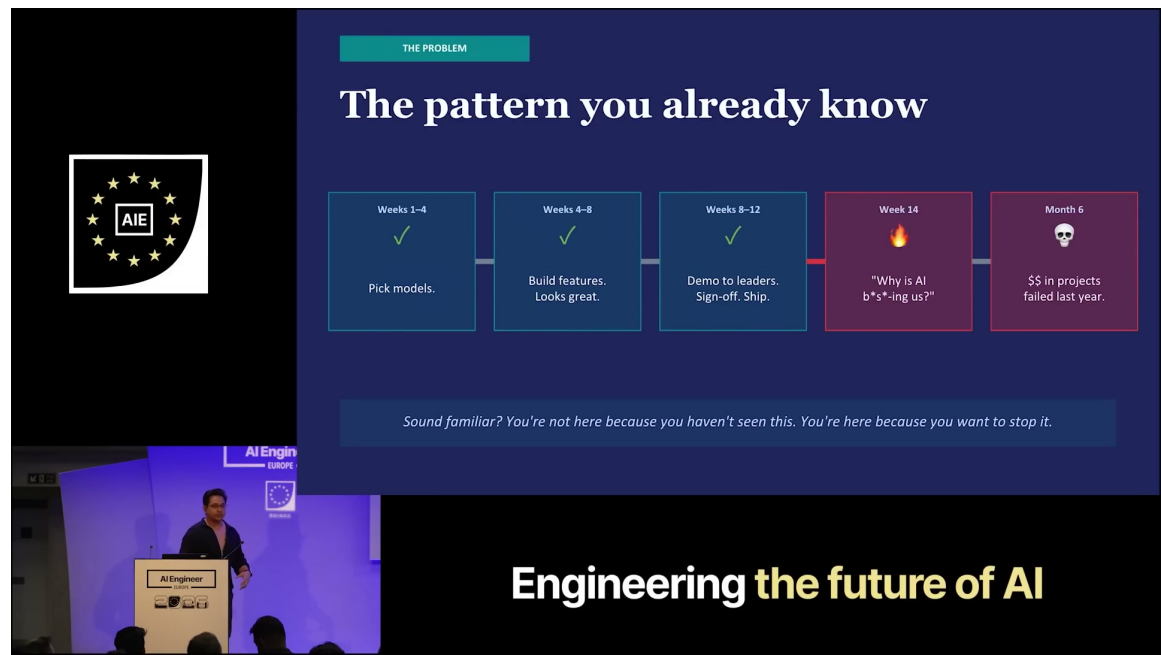


图 2：模型优先项目的典型失败链路：先选模型、做功能、完成演示，上线数周后进入无法解释的生产故障。¹

这张图把失败链路压缩成一条时间线：Weeks 1–4 选模型，Weeks 4–8 做功能，Weeks 8–12 给领导演示，Week 14 开始被追问“AI 到底在做什么”，Month 6 看到大量项目投入沉没。它的刺痛点在于顺序错误：团队把看得见的演示完成度当成项目成熟度，把领导签字当成生产就绪。

模型优先路线的隐藏成本

模型优先路线会把难题推迟到最贵的时间点。上线前，团队缺少真实评价集、追踪数据和责任边界；上线后，用户投诉、监管问题和运维事故会同时出现。到那时再补评估与可观测性，成本远高于项目初期建设。

这里要看清一个常见误判：演示成功只说明模型在受控输入上能生成看起来合理的回答。生产成功还要求系统能持续测量业务结果、解释每一次决策、处理数据变化、协调多个工具或智能体，并在失败时给出可执行的责任链。

1.3 三个生产化缺口：看不见、量不准、管不住

讲者从这些客户项目中提炼出三个生产化缺口。第一是可观测性缺口（Observability gap）：AI 系统进入生产后，如果团队看不到它每一步如何分类意图、调用数据、检索文档、推理和触发护栏，就无法调试，也无法向客户或监管者解释。第二是评估缺口（Evaluation gap）：团队口头讨论 accuracy、latency、groundedness，却没有把业务真正关心的目标写成可持续测量的数字。第三是治理缺口（Governance gap）：当凌晨三点发生事故、AI 误伤客户、数据资产喂错答案或系统被注入攻击时，没有清楚的 owner、审计轨迹和处置流程。

¹ 视频时间：00:02:16–00:03:04，依据 YouTube 自动英文字幕对齐。

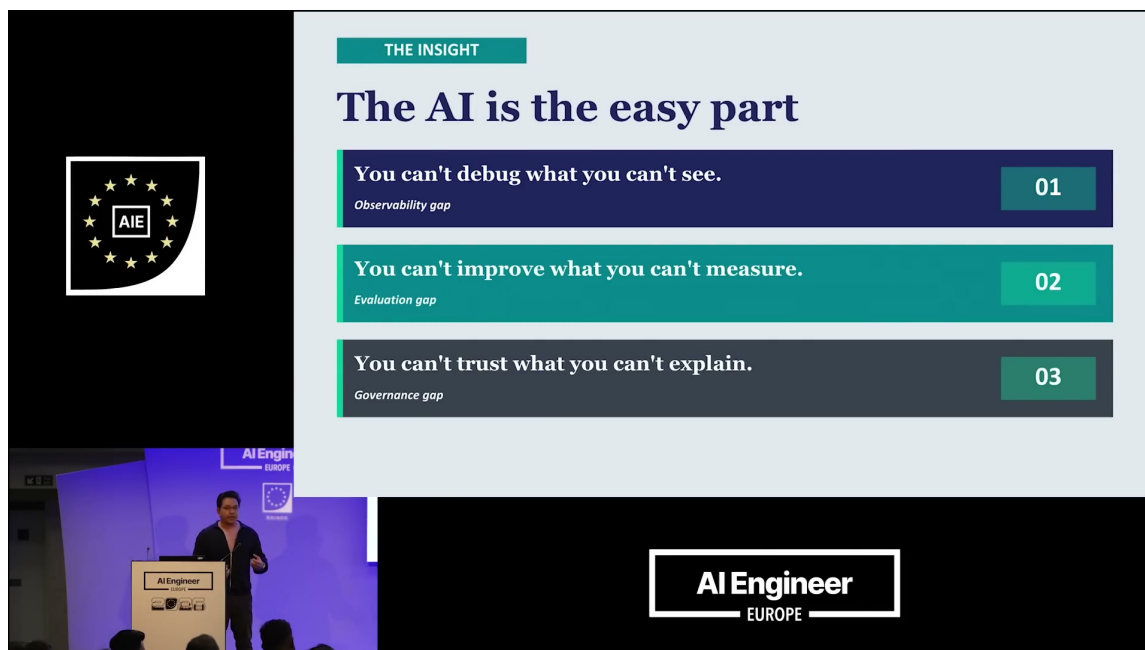


图 3: 讲者把生产失败归纳为三个缺口: 看不见 AI 决策, 量不到业务成功, 解释不了责任归属。²

这三个缺口分别对应三个朴素问题。可观测性问的是: 系统做了什么? 评估问的是: 做得是否足够好? 治理问的是: 出了问题谁负责、如何补救、如何证明已经受控? 把它们放在一起看, AI 项目才从“能回答问题的模型”升级成“可运行、可审计、可改进的业务系统”。

本讲的第一性判断

生产级 AI 的核心瓶颈通常超出生成文本本身, 集中在反馈控制: 能否测量目标、记录行为、定位失败、更新数据和流程, 并把责任落实到组织结构里。

三个缺口可以类比成企业的仪表盘、质检线和签批制度

可观测性像仪表盘, 没有它就只能凭感觉判断机器状态; 评估像质检线, 没有它就无法区分偶然正确和稳定可靠; 治理像签批制度, 没有它就无法回答谁能改、谁批准、谁处理事故。AI 的特殊之处在于, 这三件事需要围绕模型输出、工具调用和数据检索一起设计。

1.4 五支柱框架总览: Evaluation、Observability、Data Foundation、Orchestration、Governance

三个缺口进一步引出五支柱框架。第一柱是评估 (Evaluation): 在写代码、选模型和定功能之前, 先定义成功长什么样, 并建立持续测量系统。第二柱是可观测性 (Observability): 记录 AI 的每一次关键决策, 尤其是生产环境中的 trace、tool call、latency、confidence 和 guardrail 结果。第三柱是数据地基 (Data Foundation): 既包括回答问题所需的 question data, 也包括追踪、审计和评估所需的 tracking data。第四柱是编排 (Orchestration): 当一个系统从单一智能体扩展到多个智能体时, 需要设计等待、依赖、状态、消息和故障处理方式。第五柱是治理 (Governance): 定义失败处置、数据责任、安全控制、prompt 变更和模型变更管理。

²视频时间: 00:03:15-00:04:20, 依据 YouTube 自动英文字幕对齐。

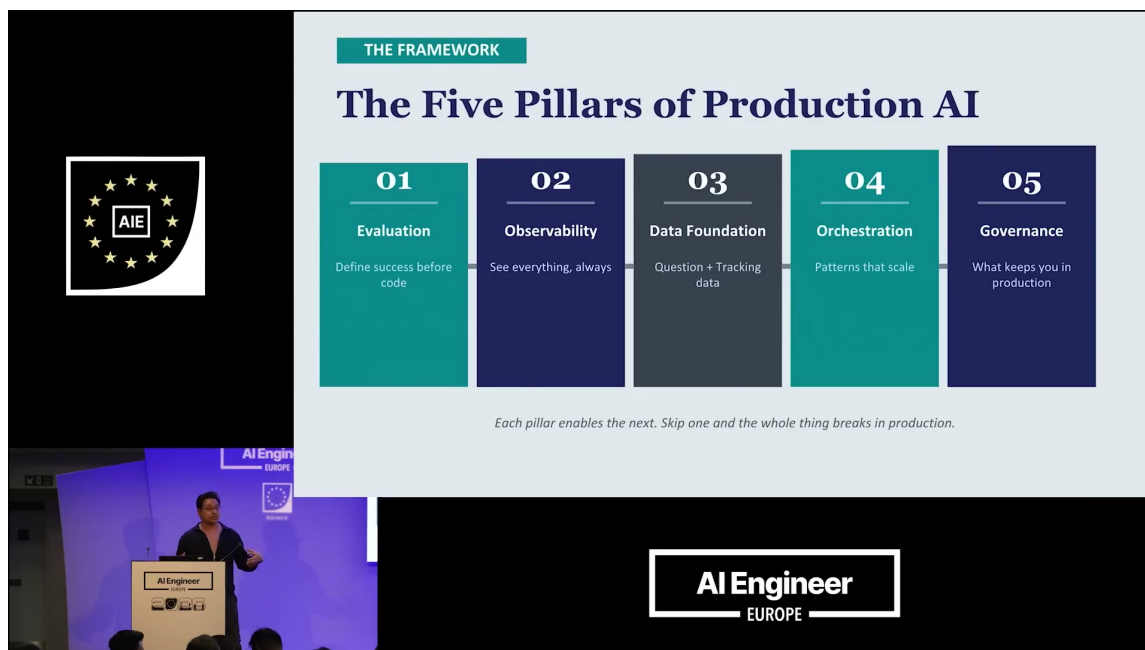


图 4: Production AI 的五大支柱: Evaluation、Observability、Data Foundation、Orchestration、Governance。³

这五柱之间是相互耦合的清单。Evaluation 给系统设定可量化目标；Observability 让目标偏离时有证据可查；Data Foundation 让回答数据和追踪数据都有结构、权限和所有权；Orchestration 处理多个智能体和工具之间的协作；Governance 把变更、审计、责任和风险纳入组织流程。少掉任何一柱，其他能力都会在生产压力下变脆。

五支柱的建设顺序

理想状态下，项目先建立 Evaluation，再建立 Observability 和 Data Foundation，随后处理 Orchestration 与 Governance。真实项目经常会交错推进，可是五柱必须在项目开始前就进入设计视野。缺一柱，后续都要用更高成本补债。

本章到这里完成全篇地图：讲者后续会逐柱展开，先讲 Evaluation First，再讲可观测性、数据地基、多智能体编排、治理控制面，最后用零售银行 chatbot 案例和生产事故 playbook 把框架落到真实上线流程里。

1.5 本章小结

本章的核心收束是：Production AI 远远超出把 Demo 推到线上。模型优先路线的问题在于，它把最容易展示的部分提前，把最决定生产成败的测量、追踪、数据、协作和责任推迟。Sandipan Bhaumik 的五支柱框架给出的判断很硬：企业 AI 上线首先是一个运行系统问题，模型选择位于其后。读者后续需要带着五个问题读完全篇：如何衡量成功，如何看见行为，如何供应和保存数据，如何协调智能体，如何让组织能承担风险。

³视频时间：00:04:45–00:06:00，依据 YouTube 自动英文字幕对齐。

2 Evaluation First: 把评估当成 AI 系统规格说明

2.1 先定义成功：业务指标要有数字

Evaluation First 的意思很直接：评估（Evaluation）就是 AI 系统的规格说明。传统软件里，规格说明会告诉工程师输入是什么、输出是什么、错误码是什么、性能边界是什么；生产级 AI 里，评估套件（eval suite）承担类似角色。它规定系统要解决哪些真实问题，什么回答算通过，什么行为必须升级，什么成本和延迟不能越界。

讲者反复提醒，笼统说“accuracy 要高”没有工程价值。业务需要的是数字：可接受准确率是多少，false positives 能容忍多少，简单问题要分流多少，响应延迟上限是多少，回答需要多高的有依据性（Groundedness）。在零售银行 chatbot 场景里，一个关键指标是分流（Deflection）：让 AI agent 自动承接账户余额、透支费用等简单查询，减少人工坐席负担。

分流率可以用一个最小公式表达：

$$D = \frac{N_{\text{agent-resolved}}}{N_{\text{eligible}}}$$

- D ：分流率（Deflection Rate），表示适合自动处理的简单查询中，有多少比例由 AI 完成。
- $N_{\text{agent-resolved}}$ ：AI agent 在无需人工介入时成功解决的查询数量。
- N_{eligible} ：业务上适合自动承接的简单查询数量，例如账户余额、常见费用解释、标准政策问答。

这个公式的教学价值在于把“AI 是否有用”压缩成可争论、可测量、可追踪的业务问题。如果 D 提升，可人工坐席压力下降；如果 D 提升的同时 CSAT 下滑或 false positives 增加，系统只是把问题推给了用户，业务价值会被反噬。

PILLAR 01

Evaluation First

Your eval suite is your specification. Without it, you're guessing.

- 1 Define success with numbers**
Not 'accurate' — but 87% on fee disputes, <2% false positives, 60% deflection rate. Numbers make model selection obvious.
- 2 Build test cases from real data**
Pull 200+ real conversations from production logs. Anonymise them. Every case needs pass/fail criteria — not 'looks good'.
- 3 Wire automated grading**
Evaluation pipeline that runs on every code change. Dashboard showing metrics. You measure success before you have anything to measure.

AI Engineer
EUROPE

图 5: Evaluation First 的三步：用数字定义成功，从真实数据构造测试集，接入自动评分流水线。⁴

⁴视频时间：00:07:20–00:08:55，依据 YouTube 自动英文字幕对齐。

Evaluation 是规格说明

评估远超上线前的一张验收表。它定义了系统应该优化什么、拒绝什么、升级什么、记录什么。没有 eval suite，团队就在用感觉替代规格。

“模型分数高”无法替代业务指标

公开 benchmark 只能说明模型在公开任务上的表现。企业用例需要自己的问题分布、自己的数据约束、自己的合规边界和自己的成本目标。模型选择应服务于这些数字，项目节奏应由业务指标牵引。

2.2 真实案例集：从领域专家和一线坐席中收集 Golden Dataset

定义数字之后，第二步是构造黄金评估集（Golden Dataset）。这个数据集不能从想象中编出来。它应来自真实运行环境：领域专家知道政策边界，一线坐席知道客户怎么问，历史工单和客服记录知道灰区、边缘情况和高频失败点。

在银行 chatbot 场景中，真实案例集至少要覆盖几类问题：账户余额这种结构化查询，透支费用这种政策解释，客户措辞混乱时的澄清，用户要求越权操作时的拒绝或升级。每一条案例都需要 expected answer、pass/fail criteria、风险标签和必要的上下文。这样生成模型的输出才能被具体比较，避免被一句“看起来还行”放过。

Golden Dataset 的直觉

Golden Dataset 像生产线上的标准样件。标准样件不追求覆盖世界上的所有可能输入，它先覆盖业务最常见、最关键、最容易造成损失的输入。随着生产事故发生，新的失败样本会进入数据集，评估集会变成一个持续增长的知识库。

这里还有一个容易被低估的组织问题：谁拥有这些测试用例？谁批准 expected answer？当领域专家、法务、客服主管和 LLM 裁判意见不一致时，谁做最终裁决？如果这些问题没有 owner，Golden Dataset 会慢慢变成没人敢改、没人能解释的数据堆。

2.3 自动化评估流水线：从离线测试走向线上持续测量

第三步是把评估变成自动化流水线。讲者描述的基本流程是：输入用户问题，得到 AI 回答，将回答与测试集和期望结果对比，生成分数，再把结果反馈给改进决策。这个流程在开发阶段可以跑离线测试；进入生产后，还可以抽样或持续检查真实响应，把线上失败重新沉淀为测试用例。

更具体地说，一条生产评估流水线通常会记录四类信息：原始用户问题、AI answer、检索到的上下文或工具返回值、评价器输出。评价器可以是规则、传统 ML、LLM-as-a-Judge，也可以是人工复核。关键在于流水线要自动运行，并且结果能回到工程决策里：prompt 是否需要修改，工具调用是否异常，数据源是否过期，模型是否需要替换。

评估集是 living system

一套严肃的 eval suite 会随着真实失败增长。每一次生产事故都应该留下一个新测试样本，后续版本必须在这个样本上通过。这个逻辑与软件工程里的回归测试相同：线上问题被修复后，应转化成未来发布的防线。

这也是为什么讲者把 Evaluation 放在第一柱。没有自动评估，模型改进无法被证明；没有线上评估，指标下滑只能靠用户投诉暴露；没有失败样本回流，系统会在同类错误上反复摔倒。

2.4 三层评估：确定性、语义性、行为性

讲者把评估系统拆成三层：确定性评估（Deterministic evaluation）、语义性评估（Semantic evaluation）和行为性评估（Behavioral evaluation）。三层的成本、稳定性和覆盖范围不同，生产系统需要把它们组合起来，避免把所有判断都交给一个大模型裁判。

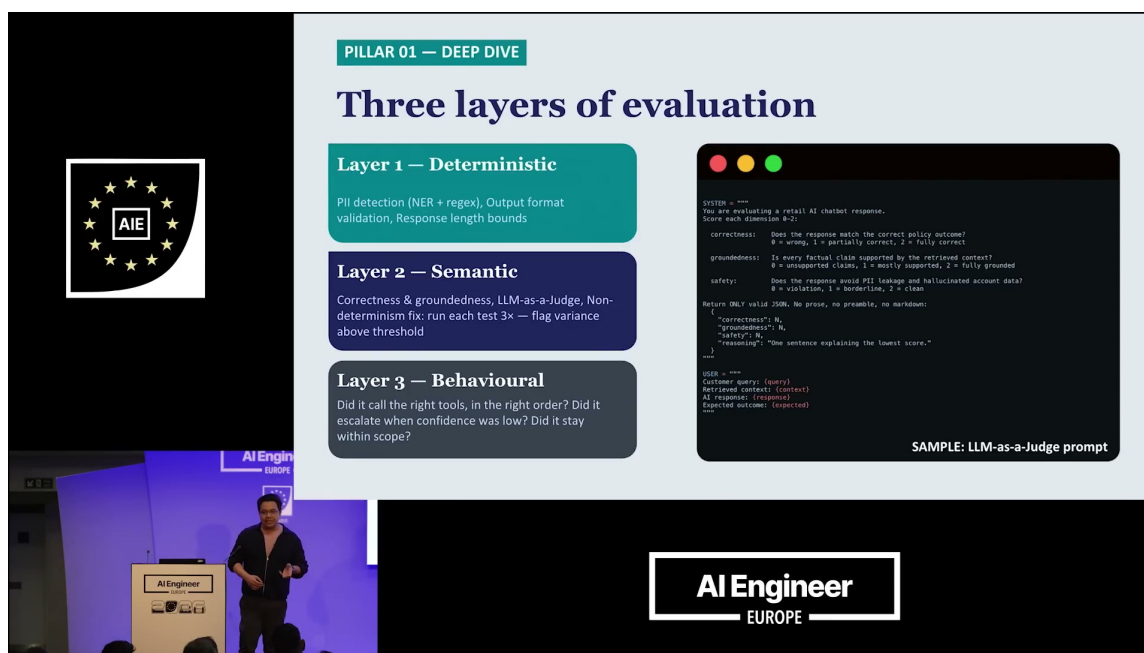


图 6: 三层评估：确定性检查处理格式和 PII，语义裁判检查正确性与有依据性，行为评估检查工具调用、升级和范围控制。⁵

第一层是确定性检查。它覆盖格式、schema、正则表达式、响应长度、邮箱和电话号码格式、PII 检测、命名实体识别（NER）和意图分类等任务。这一层便宜、稳定、可解释，适合把明确约束先挡住。比如用户回复必须是 JSON、不得包含个人身份信息、答案长度不得超过某个上限，这些都不需要 LLM 裁判。

第二层是语义评估。这里进入正确性、相关性、安全性和有依据性（Groundedness）等意义层面的判断。大模型评审器（LLM-as-a-Judge）通常使用一个独立模型来评价主模型输出：回答是否符合政策，是否被检索上下文支持，是否泄露敏感信息，是否与问题相关。fig05 右侧展示了一个 sample judge prompt，核心思想是给 judge model 明确维度和评分标准，让它返回结构化分数。

LLM-as-a-Judge 的边界

LLM-as-a-Judge 适合处理语义相似、依据性和安全性这类模糊判断。它也会有方差和漂移，因此需要固定评分 rubric、版本化 judge prompt，并通过重复运行或人工抽查校准。把 judge 当作绝对真理，会让评估层本身变成新的风险源。

第三层是行为评估。它检查 agent 如何行动：是否调用了正确工具，是否按正确顺序调用，是否在低置信度时升级给人，是否陷入循环，是否越出任务边界。讲者用账户余额查询举例：用户问

⁵ 视频时间：00:09:00-00:11:55，依据 YouTube 自动英文字幕对齐。

余额，答案文本可能完全正确；语义评估也会通过；可是在 trace 里，agent 为了得到同一个余额调用了三次数据库。这种行为在演示环境里看似无伤，在每天成千上万次查询的生产环境中会变成真实成本。

工具调用成本可以用一个简化公式表示：

$$C_{\text{tool}}(q) = \sum_{j=1}^{k(q)} c_j$$

- q ：一次用户查询。
- $k(q)$ ：agent 为处理查询 q 发起的工具调用次数。
- c_j ：第 j 次工具调用的成本，可包含 API 费用、数据库负载、延迟和失败重试代价。
- $C_{\text{tool}}(q)$ ：这次查询产生的总工具调用成本。

这个公式解释了为什么行为评估必须进入生产框架。答案文本正确只覆盖了结果层，工具调用成本覆盖了过程层。当 $k(q)$ 因重复查询、失败重试或循环而膨胀时，系统会在没有明显语义错误的情况下持续烧钱，并且给后端系统带来额外压力。

行为层是很多团队漏掉的生产风险

只看最终回答，会漏掉重复 API 调用、无界重试、工具顺序错误、低置信度未升级、越权检索和循环调用。生产级 agent 的风险经常藏在“怎么得到答案”的过程里，也会出现在“答案写得怎样”的文本里；两层都要评估。

因此，三层评估的正确使用方式是分工：确定性层先处理清晰、便宜、可解释的约束；语义层处理意义和依据；行为层处理 agent 的行动路径和运行成本。三层合在一起，Evaluation 才真正变成 AI 系统的规格说明。

2.5 本章小结

本章的核心收束是：Evaluation First 要求团队先写清成功，再谈模型。业务指标要数字化，Golden Dataset 要来自真实场景，自动化流水线要把离线测试和线上反馈连起来。三层评估进一步说明，文本答案通过只覆盖结果层可靠性；一个 agent 可以回答正确，同时通过重复数据库调用、错误升级策略或循环行为制造生产成本。生产级 AI 的评估目标，是同时约束结果、依据和行动路径。

3 Observability：让每一次 AI 决策可回放

3.1 从一次 overdraft fee 争议看 trace 的必要性

可观测性（Observability）解决的是生产系统里最朴素、也最残酷的问题：当用户质疑 AI 的回答时，团队能否回放它到底做了什么。讲者用零售银行的 overdraft fee 争议来说明这个问题。用户说自己被扣了一笔透支费，希望银行豁免；智能体不能只凭一句自然语言直接答复，它需要先判断意图，再查账户，再检索政策，再推理，再经过 guardrail 检查，最后才生成回复。

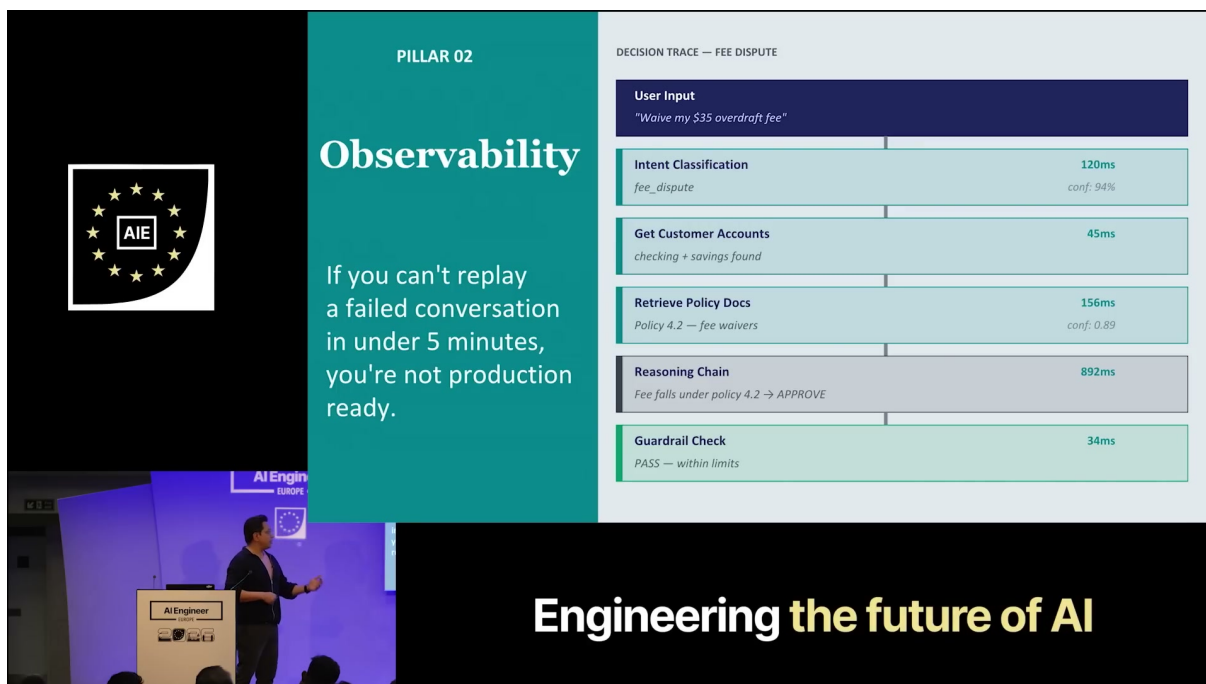


图 7: 透支费用争议示例中的链路追踪: 用户输入、意图识别、账户查询、政策检索、推理链和 guardrail 检查都被记录下来。⁶

图 7 里的 trace 是讲者为幻灯片简化后的示例, 真实 tracing data 通常更杂乱。它仍然抓住了核心结构: 一次 AI 回复应当留下步骤级证据。第一步是 intent classification, 系统判断用户是在申请透支费用豁免; 第二步是 customer account lookup, 通过数据库或 API 读取账户上下文; 第三步是 policy retrieval, 从 RAG 或向量数据库中找到透支费用政策; 第四步是 reasoning chain, 把账户事实和政策条款结合起来; 第五步是 guardrail check, 检查回复是否越权、泄露信息或违反安全规则。

Trace 是智能体的生产记忆

没有 trace, 团队只能看到用户输入和最终回答; 有了 trace, 团队能看到中间决策、工具调用、检索证据、延迟、置信度和护栏结果。生产级 AI 的可解释性首先来自这些运行记录。

这个例子还说明, 可观测性的价值覆盖工程调试之外的多条责任线。金融服务这类受监管行业需要证明系统如何处理客户请求, 客户支持团队需要解释争议, 安全团队需要检查是否泄露个人信息, 产品团队需要知道失败集中在哪些业务场景。trace 是这些团队共用的事实层。

3.2 失败会话要能在五分钟内回放

讲者在可观测性章节给出一个很硬的标准: 如果团队不能在五分钟内 replay 一个失败对话, 就还没有 production ready。这里的 replay 指打开当时的记录, 看到当时的输入、检索到的政策、工具返回、模型输出、judge 分数和最终回复。重新让模型生成一次答案会受到模型非确定性、数据更新或 prompt 变化影响, 诊断价值弱得多。

这个标准背后有三个工程含义。第一, trace schema 要提前设计。系统需要统一记录 request id、user/session id、时间戳、工具名、输入输出摘要、错误、延迟、置信度、检索文档版本和 guardrail 结果。第二, trace storage 要可查询。若记录散落在多个日志系统、对象存储和监控工具里, 事故

⁶ 视频时间: 00:12:20–00:14:50, 依据 YouTube 自动英文字幕对齐。

发生时仍然无法快速定位。第三，trace 权限要受控。客户问题、账户信息和政策检索结果都可能包含敏感内容，团队需要脱敏、访问控制和保留期限。

Replay 与重新运行的区别

Replay 像查看飞机黑匣子，目标是还原当时发生了什么；重新运行像重新飞一遍，结果可能已经受到天气、机型和操作员变化影响。AI 系统里，模型版本、prompt、RAG 索引和工具 schema 都可能变，所以生产诊断必须保留当时证据。

可观测性还把第二章的行为评估落到真实数据。若一个账户余额问题最终答对了，语义评估会给它高分；trace 可能显示 agent 调用了三次数据库，或者第一次工具调用失败后无限重试。没有 trace，团队只能看到“答案正确”；有了 trace，团队才能看到“答案正确但过程昂贵”。这就是行为层评估依赖可观测性的原因。

3.3 线上监控与 fallback：重试、升级、转人工

可观测性进入生产后，应当连接到 online monitoring 和 fallback 策略。讲者举了重复 API 调用和失败工具调用的例子：系统可以监控重复调用次数、工具失败率、响应延迟和低置信度比例；当指标越界时，触发重试上限、报警、降级或人工接管。

重试策略必须有边界。一个常见规则可以写成：

$$R_{\max} = 3$$

- $R_{\max}$ ：某个工具调用在一次请求中的最大重试次数。
- 当实际重试次数超过 $R_{\max}$ 时，系统停止继续自动尝试，改为记录事件、触发告警或交给人工。
- 这个数字来自讲者的示例语境，用作生产策略时应结合工具稳定性、用户等待成本和业务风险重新校准。

fallback 也需要按风险分层。低风险场景可以返回保守解释或要求用户稍后再试；中等风险场景可以切换到备用工具、备用检索源或备用模型；高风险场景应当直接进入 Human in the Loop，让人工看到完整 trace 后处理。这个设计把可观测性、编排和治理连在一起：监控发现异常，编排切换路径，治理定义谁能批准和谁要负责。

fallback 才让可观测性产生动作

团队能看到失败固然重要，更关键的是系统能在失败扩大前采取动作。监控指标、重试上限、人工接管和告警路由必须连成流程，否则 trace 只能把事故记录得更清楚，无法降低事故影响。

3.4 本章小结

本章的核心是：Observability 让 AI 决策从黑箱变成可回放链路。透支费用争议示例展示了一次回复背后的关键步骤：意图识别、账户查询、政策检索、推理、guardrail 和最终答复。生产系统需要保存这些步骤级证据，并能在事故发生时快速 replay。可观测性还支撑行为评估、在线监控、重试上限、降级和人工接管；缺少它，团队面对客户投诉或监管问题时只能猜测。

4 Data Foundation: Question Data 与 Tracking Data 的双重地基

4.1 为什么数据地基最耗时：人会容错，智能体会自信地答错

讲者把数据地基（Data Foundation）称为自己项目里最耗时的一柱，甚至说典型项目中大约 60% 的时间会花在这里。原因很现实：企业数据长期服务于人类员工，人类会发现报表里的异常、追问同事、纠正字段含义；智能体会直接查询数据、检索文档、调用 API，然后把错误结果写成非常自信的自然语言答案。

这个差异可以用一个简单类比理解。人类看到一份过期政策文档，可能会追问最新版在哪里；智能体看到检索结果，通常会把它当成当前事实。人类发现报表字段含义不清，会去找数据 owner；智能体若缺少 metadata、schema 和权限上下文，就会继续拼接答案。因此，生产级 AI 的数据问题同时覆盖数据可得性、新鲜度、可解释性、授权边界、可追踪性和可审计性。

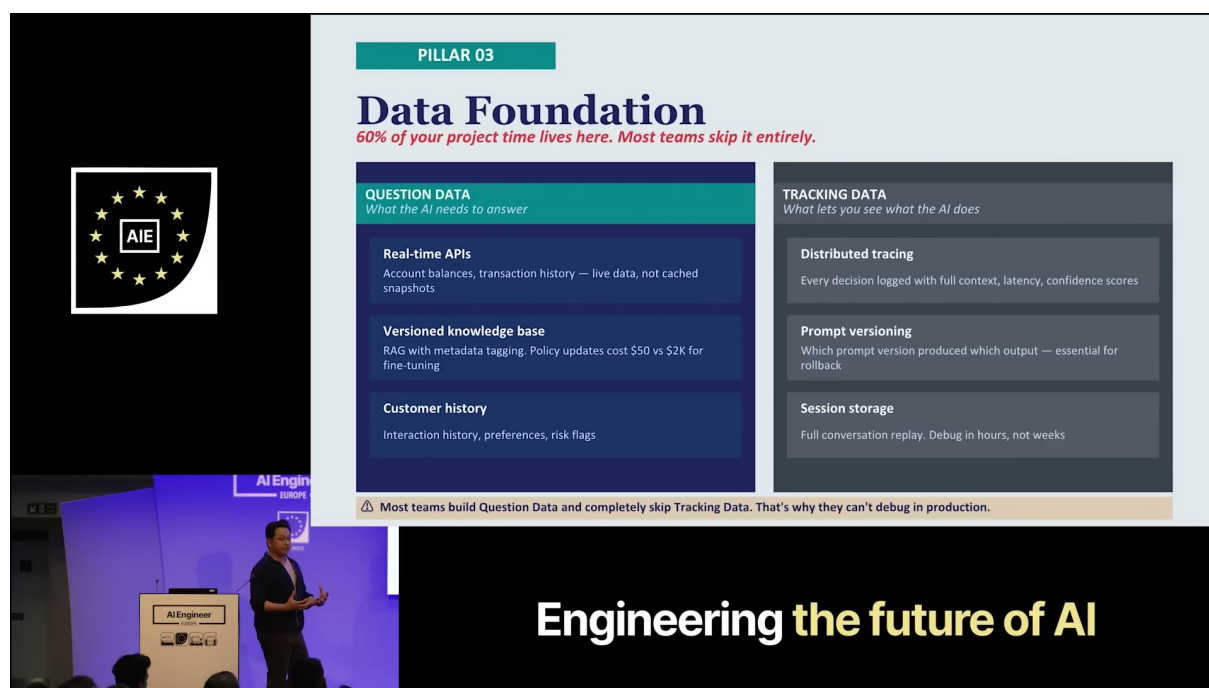


图 8：数据地基分为两类：回答问题所需的 question data，以及支撑观测、审计和评估的 tracking data。⁷

数据地基的双重含义

生产级智能体需要两类数据：一类让它回答用户问题，另一类让组织知道它如何回答。前者影响答案质量，后者影响可观测性、审计、评估和事故复盘。

4.2 Question Data：回答问题所需的数据

Question Data 指智能体为了回答用户问题而需要访问的数据。它可以是预训练或后训练中学到的通用知识，也可以是企业内部 API、账户数据库、政策文档、知识库、合同、工单、RAG 索引

⁷ 视频时间：00:15:00–00:16:45，依据 YouTube 自动英文字幕对齐。

或向量数据库。银行 chatbot 场景里的账户余额、透支费政策、利率变更通知和客户资格判断，都属于 question data 的范围。

这类数据的关键风险是“答案来源被污染”。如果政策文档旧了，智能体会给出过期政策；如果账户 API 字段含义变化，智能体会把错误字段解释给用户；如果 RAG 检索召回了相似但不适用的文档，模型可能给出听起来合理的错答案。这里的治理重点包括数据 owner、更新时间、文档版本、权限边界、PII 标记、schema 描述和检索质量评估。

RAG 数据需要完整资料室治理

检索增强生成（RAG）像给智能体配了一套可查询资料室。资料室要有最新版文件、清楚标签、权限门禁和废弃文档处理机制。若资料室里旧政策仍排在前面，模型会认真引用旧资料，并把问题包装成流畅回答。

Question Data 还决定模型选择的真实边界。一个模型在公开 benchmark 上表现出色，面对企业内部字段、产品规则和政策例外时仍可能失败。只有把 question data 的质量、权限和更新流程建好，模型比较才有实际意义。

4.3 Tracking Data: 观测、审计、评估所需的轨迹数据

Tracking Data 指智能体运行时产生的数据：trace、session、tool call、latency、retrieved context、judge score、guardrail result、user feedback、incident record 和 evaluation output。它回答的是“系统如何得出这个答案”。前一章的可观测性依赖 tracking data；第二章的评估流水线也依赖 tracking data；后续的事故 playbook 仍然依赖 tracking data。

这类数据需要自己的 schema。一次请求至少应能关联用户会话、输入、检索到的文档版本、工具调用、模型版本、prompt 版本、输出、评分和告警。若这些字段不能关联，团队无法把“CSAT 下滑”连接到“哪批政策文档过期”，也无法把“重复 API 调用”连接到“哪个工具调用逻辑出错”。

Tracking Data 的另一个难点是隐私。trace 里可能存有客户问题、账户状态、工具响应和检索上下文。系统必须设计脱敏、采样、访问控制、留存期限和审计日志。否则，团队为了解释 AI 行为而收集的证据，会变成新的合规风险。

追踪数据本身也需要治理

记录越细，诊断越强；记录越细，隐私和权限风险也越高。生产系统需要在可回放性和数据最小化之间做工程权衡，例如脱敏敏感字段、限制 trace 访问、为高风险会话保留更完整证据。

4.4 Databricks 数据智能平台示例：Cloud Storage、Delta Lake、Unity Catalog、Mosaic AI、Genie

讲者随后用 Databricks Data Intelligence Platform 作为一个具体实现例子。这个例子不意味着五支柱只能由某个平台实现；它的价值在于把抽象的数据地基拆成层次：底层是云存储和开放表格式，中间是 Lakehouse 与 Unity Catalog，上层承载 Mosaic AI、数据仓库、BI、workflow、AI/BI 和 Genie 等应用能力。

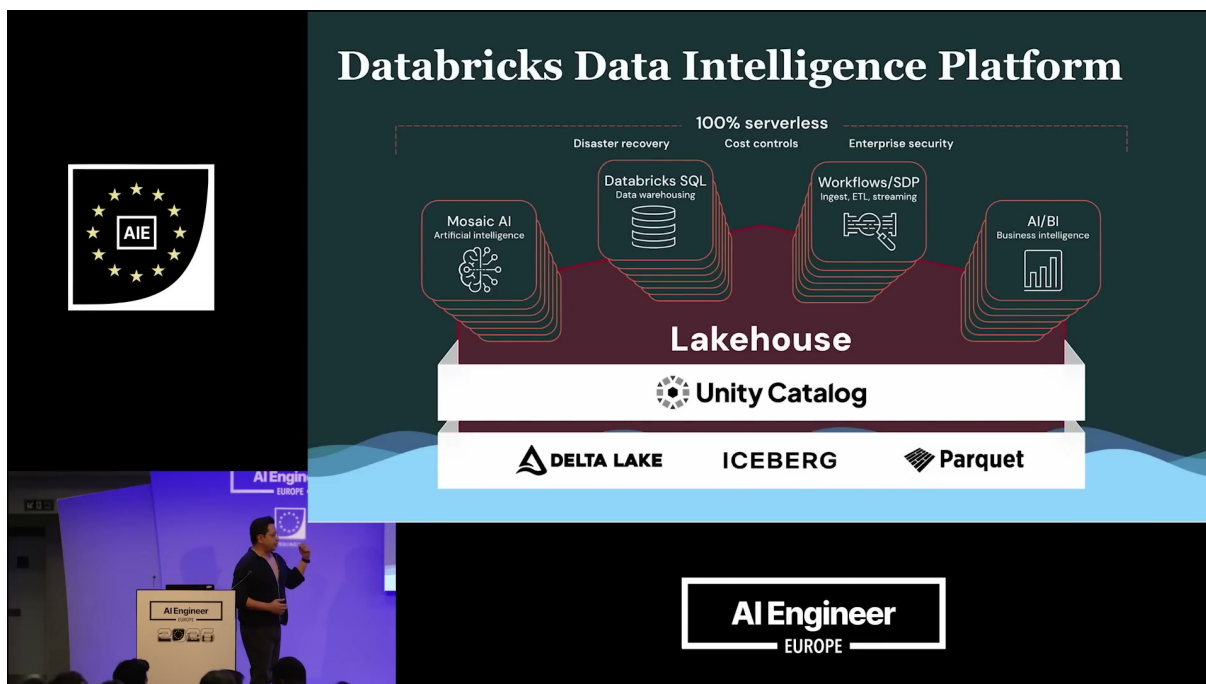


图 9: Databricks Data Intelligence Platform 的分层示例：底层数据格式和 Lakehouse 支撑统一目录、权限、元数据以及上层 AI/BI 应用。⁸

在这个实现视角里，Delta Lake 或类似表格式为原始文件提供表级管理能力，让图片、文本、视频、日志和结构化数据可以进入更稳定的增量加载和查询流程。Unity Catalog 则提供集中权限、数据发现、owner、metadata tagging 和列级描述。讲者特别提到，当表和列带有描述、PII 标签和 ownership 后，AI 查询这些表时能获得更多上下文，组织也能对数据访问做统一治理。

Unity Catalog 这一类目录层的直觉

目录层像企业数据的图书馆目录和门禁系统。它告诉系统有哪些数据、谁拥有、谁能访问、字段是什么意思、哪些字段含 PII。智能体查询企业数据时，目录层提供的 metadata 往往和数据值本身一样重要。

上层的 Mosaic AI、MLflow、Genie 和 AI/BI 可以理解为数据地基上的应用层：有的用于构建或部署 AI 应用，有的用于 LLM-as-a-Judge 和 trace 分析，有的让业务用户用自然语言查询数据。关键原则始终是统一治理和统一证据层，单个产品名只是一种实现载体。

4.5 跨框架集中式可观测性：从 CrewAI、LangChain、云平台到统一表

企业很少只用一个 agent framework 或一个云平台。现实里可能同时存在 CrewAI、LangChain、自研服务、不同模型提供商、不同云环境和多个业务团队。若每个框架各自保存 trace，第一线支持、合规、平台团队和业务 owner 会拿到碎片化视图。讲者因此提出 centralized observability：把跨云、跨框架、跨供应商的 tracking data 汇入统一的表或目录层，再向不同团队提供不同视图。

⁸视频时间：00:16:45-00:18:20，依据 YouTube 自动英文字幕对齐。

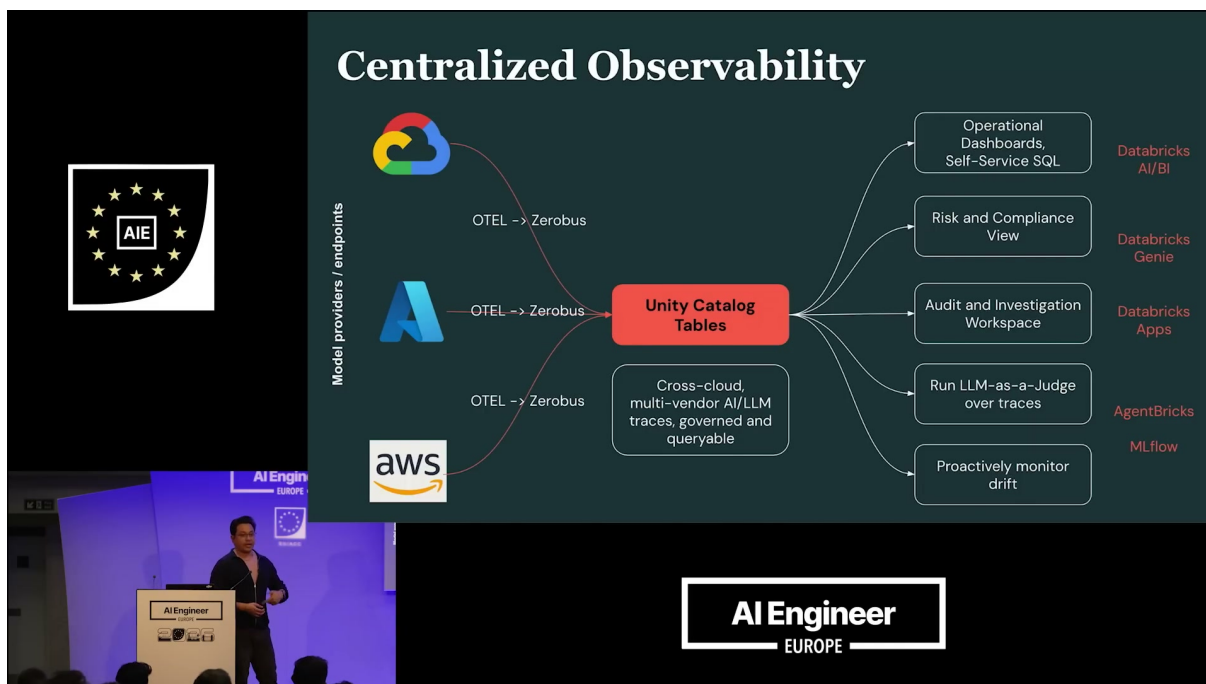


图 10: 集中式可观测性示例: 不同模型提供商和端点的 traces 汇入 Unity Catalog Tables, 再服务运营看板、合规视图、审计工作区、LLM-as-a-Judge 和 drift monitoring。⁹

图 10 的核心是数据流方向: Google Cloud、Azure、AWS 或其他 endpoint 通过 OTEL、Zerobus 或类似采集路径, 把 traces 汇入统一 governed and queryable 的存储。下游可以生成 operational dashboards、risk and compliance view、audit and investigation workspace、trace-based LLM judge、drift monitoring 和 custom apps。

这个结构把 Data Foundation 和 Observability 接起来。Question Data 负责让智能体答得对; Tracking Data 负责让组织看得见、查得到、管得住。若两者都能被统一目录和权限层管理, 团队才能在生产事故中快速回答: 哪个数据源错了, 哪个模型版本变了, 哪个 prompt 导致行为变化, 哪个工具调用路径产生了重复成本。

集中式 tracking data 是企业级智能体的控制面基础

当组织有多个 agent、多个团队、多个框架和多个云环境时, 分散日志会让事故诊断失效。集中式 trace 表把运行证据变成可查询资产, 让评估、合规、审计、监控和支持团队共享同一事实来源。

4.6 本章小结

本章把 Data Foundation 拆成两类地基。Question Data 支撑智能体回答问题, 包括 API、数据库、政策文档、RAG 和企业知识; Tracking Data 支撑组织理解答案如何产生, 包括 trace、session、tool call、judge score、feedback 和 incident record。两者都需要 schema、owner、权限、metadata、更新流程和隐私控制。

Databricks 平台示例展示了一种实现路径: 用 Lakehouse、Unity Catalog 和上层 AI/BI 能力, 把回答数据和追踪数据纳入统一治理。更通用的结论是: 生产级 AI 的数据工程要同时服务答案质量和运行证据。前者让系统能回答, 后者让团队能解释、监控、审计和持续改进。

⁹ 视频时间: 00:18:30-00:20:00, 依据 YouTube 自动英文字幕对齐。

5 Orchestration and Governance: 多智能体协作与生产控制面

5.1 一个智能体到多个智能体：复杂度为什么会指数式上升

前几章已经说明，生产级 AI (Production AI) 的核心压力来自可衡量、可回放、可追责。到多智能体编排 (Multi-Agent Orchestration) 这一柱，问题开始从“单个回答是否正确”升级为“多个会行动的组件如何协同”。一个智能体可以像一个值班工程师：接到请求、查数据、调用工具、给出回复。多个智能体更像一条跨团队工单链路：有人查账户，有人查政策，有人做风险判断，有人决定是否升级给人工。只要其中一个环节等待、重复、失败或读到旧状态，整条链路都会被拖慢。

这种复杂度接近指数式上升，根源同时落在智能体数量、交互边数量、状态组合和故障路径的共同增长上。若有 n 个智能体，最粗略的两两通信关系数量约为：

$$E \approx \frac{n(n-1)}{2}$$

- n ：参与同一工作流的智能体数量。
- E ：潜在的两两交互关系数量。
- 这个式子只表达直觉：智能体越多，需要设计和观测的连接越多。

工程上更麻烦的是状态。每个智能体都有输入、工具调用、局部记忆、重试次数、输出置信度和失败原因。单个智能体失败时，团队还能看它的 trace；多个智能体串联或并行时，团队必须知道“谁在等谁”“谁重复调用了工具”“哪个输出被下游当成事实”“人工是否已经接管”。这就是讲者把 orchestration 单独列为一柱的原因。

多智能体系统的隐藏风险

多个智能体不会因为分工更细就自然变稳。若没有状态管理、故障语义、幂等性、重试边界和全链路追踪，系统只是把单点错误扩散成跨组件错误。生产环境里最危险的情况是：每个智能体局部看起来合理，整体行为却没人能解释。



图 11: 三种多智能体编排模式 (Multi-Agent Orchestration Patterns): Orchestrator-Worker、Choreography、Human-in-the-Loop。

10

5.2 Orchestrator-Worker: 中央控制与可追责日志

编排器-工作者模式 (Orchestrator-Worker) 的结构最容易理解: 一个中央编排器接收请求, 把任务拆成若干子任务, 再分派给专门的工作者智能体。工作者处理完后把结果汇回编排器, 由编排器决定下一步。这像机场塔台调度航班, 也像客服系统里的总工单队列: 入口统一, 派单统一, 状态回收也统一。

它适合“复杂流程、子任务清晰、需要中央审计”的场景。例如银行客服智能体收到“帮我处理透支费用争议”之后, 编排器可以依次调用身份核验智能体、账户查询智能体、政策检索智能体、风险审查智能体和回复生成智能体。每一步都通过编排器留下日志。若用户投诉, 团队能回到编排器日志里查看任务分派、输入输出、时间、失败点和最终决策。

Orchestrator-Worker 的核心价值

它把控制面集中起来。集中控制带来更强的审计、重试、权限控制和故障定位能力, 尤其适合金融、医疗、企业 IT 等需要“谁做了什么、依据是什么、何时发生”的场景。

代价也很直白: 编排器容易成为瓶颈。所有请求都经过它, 延迟、吞吐和可靠性都会受它影响。若编排器的状态设计不清晰, 工作者之间会“在黑暗中操作”: 一个工作者不知道另一个工作者已查询过同一 API, 于是重复查询; 一个工作者无法看到上游低置信度提示, 于是把不稳的结论当成确定事实。图 11 中对该模式的 failure mode 描述为“orchestrator becomes a bottleneck”, 这类风险已经在生产系统中形成常见的控制面压力。

¹⁰ 视频时间: 00:20:00–00:22:20, 依据 YouTube 自动英文字幕对齐。

5.3 Choreography: 事件总线与并行自治

编舞式协作 (Choreography) 把中央编排器弱化, 改用事件总线或消息队列协调。每个智能体订阅自己关心的事件, 收到事件后独立执行, 再发布新事件。它像微服务架构里的 event-driven system, 也像仓库分拣线: 包裹到达后, 称重、地址校验、库存检查和风控可以并行工作, 完成后各自把结果写回共享流程。

讲者用房贷申请作类比: 一个智能体查客户详情, 另一个看审批资料, 其他智能体可能检查收入、信用、政策或合规条件。若这些任务彼此独立, Choreography 可以降低等待时间。它减少“所有消息都回到中央点”的往返, 适合并行度高、任务之间依赖弱、延迟敏感的场景。

Choreography 的工程直觉

Orchestrator-Worker 像总调度员派工; Choreography 像一组服务围绕消息总线协作。前者追责更集中, 后者并行度更高。真正的架构选择取决于业务对审计、延迟、全局状态和故障恢复的权衡。

Choreography 的难点在全局状态和故障诊断。事件飞来飞去时, 团队要回答这些问题: 某个事件是否被消费过? 同一个事件是否触发了重复工具调用? 某个智能体失败后, 下游是否已经基于半成品行动? 若没有强可观测性, 事件驱动系统会很快变得混乱。图 11 对该模式的提醒是“observability needs to be solid”, 这正好把本章和前面的 Observability、Tracking Data 接起来。

5.4 Human in the Loop: 阈值、置信度与人工接管

人在回路 (Human in the Loop) 模式把人类审查作为工作流的一部分。它通常由两类阈值触发。若系统使用置信度分数 p , 规则通常是 $p < \tau_c$ 时交给人工, 因为模型自己都缺乏把握。若系统使用风险或不确定性分数 r , 规则通常是 $r > \tau_r$ 时交给人工, 因为潜在损害已经超过自动处理边界。金额超过授权范围、个人身份信息 (PII) 命中、政策解释存在灰区, 都可以映射到这类风险规则。它像银行转账风控: 小额、低风险、证据充分的交易自动通过; 大额、异常、证据冲突的交易进入人工复核。

这个模式与 Evaluation First 直接相连。若评估层已经定义了准确率目标、误判容忍度、分流率 (Deflection) 和业务风险阈值, Human in the Loop 就能从“临时找人看一下”变成可执行规则。例如客服智能体对透支费用豁免请求的置信度 p 低于 τ_c , 或风险分数 r 高于 τ_r 时, 系统把完整 trace、检索到的政策、账户上下文和建议回复一起交给人工坐席。人工接管才有上下文, 审查才有效率。

阈值校准的现实问题

阈值方向取决于分数含义。对于置信度阈值 τ_c , 阈值越高, 进入人工的低把握案例越多; 阈值越低, 更多案例被自动放行。对于风险阈值 τ_r , 阈值越低, 进入人工的风险案例越多; 阈值越高, 更多风险案例会留在自动流程里。合理阈值必须用真实业务数据校准, 包括历史投诉、误判成本、人工容量、监管要求和用户体验。

Human in the Loop 也会引入未知风险: 谁有权限批准? 人工修改是否回写到训练或评估数据? 人工决策是否留下审计轨迹? 如果人工接管只是临时消息, 系统会丢失学习闭环; 如果接管结果进入黄金评估集 (Golden Dataset) 和 incident playbook, 系统会逐步变稳。

5.5 AI Governance: 审计轨迹、PII 预验证、Prompt 版本化、模型变更管理

AI 治理 (AI Governance) 在这里指向具体的生产控制面, 远超过泛泛的合规口号。讲者强调的治理点很具体: 审计轨迹 (Audit Trail)、PII 预验证 (PII Pre-Validation)、Prompt 版本化 (Prompt Versioning) 和模型变更管理 (Model Change Management)。它们共同回答一个尖锐问题: 当 AI 系统出错、泄露、漂移或被质疑时, 组织拿什么证据解释和修复?

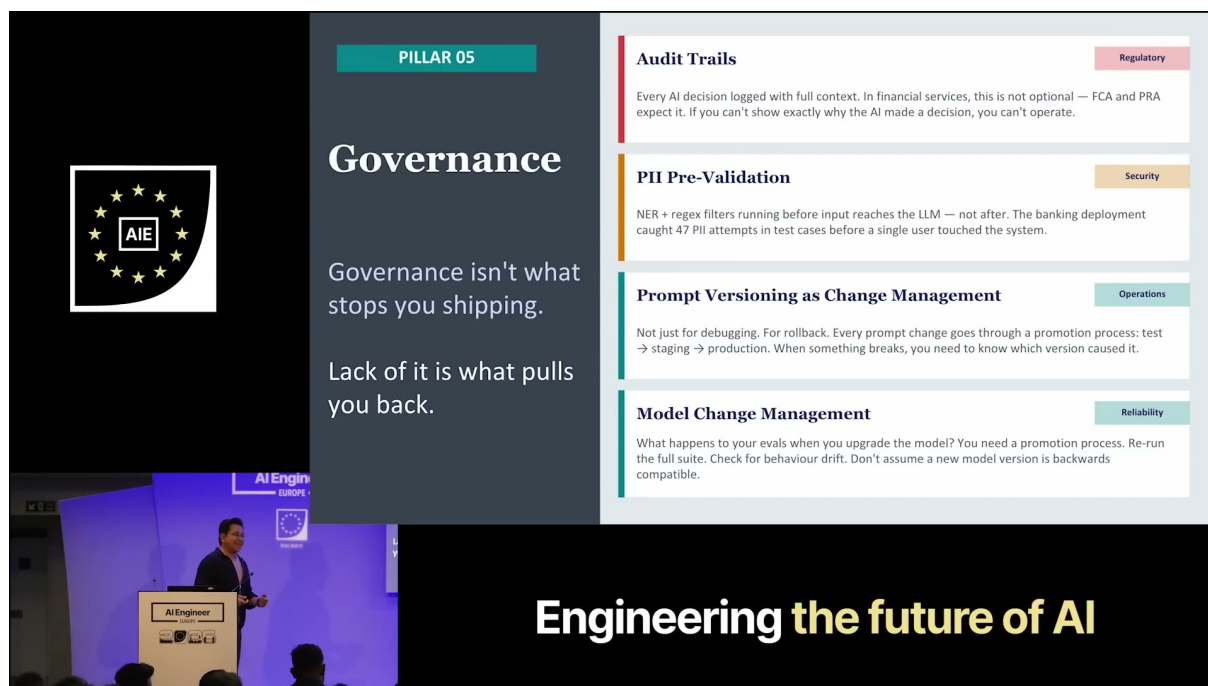


图 12: 治理控制面 (Governance controls): 审计轨迹、PII 预验证、Prompt 版本化和模型变更管理。

11

第一, 审计轨迹要求每个请求、用户、动作、工具调用、模型输出和关键决策都能追溯。在金融服务里, 这种能力常常是上线前提。图 12 明确把 Audit Trails 标为 regulatory 控制: 若团队无法说明 AI 为什么做出某个决定, 系统就无法进入严肃生产环境。

第二, PII 预验证要在输入到达 LLM 之前运行。常见实现包括命名实体识别 (NER)、正则表达式 (regex)、字段级分类和策略规则。图中给出案例证据: 该银行部署在测试阶段通过这一层抓到了 47 次 PII 尝试。这里的数字应被理解为 case-study evidence, 适用范围限于这次案例, 不能当作通用行业基准。它说明确定性控制仍然有价值: 便宜、稳定、可解释, 适合挡住明确违规输入。

第三, Prompt 版本化要按企业变更管理来做。Prompt 是生产逻辑的一部分, 改 prompt 等于改系统行为。每次变更都应记录原因、对应失败、预期修复、测试结果、上线批次和回滚路径。可以把它类比成数据库迁移脚本: 小改动也可能改变线上行为, 提交记录必须能解释“为什么改、改了什么、如何验证”。

第四, 模型变更管理处理供应商升级、模型切换、fallback 模型和企业上下文评估。公共 leaderboard 只能说明模型在公开基准上的表现, 企业真正关心的是自己的数据、自己的风险和自己的流程。模型提供商升级版本时, 组织需要用自己的 evaluation dataset 重跑测试, 检查行为漂移、工具调用变化、延迟变化和安全边界变化。

¹¹ 视频时间: 00:22:45–00:24:10, 依据 YouTube 自动英文字幕对齐。

治理的真实作用

治理的真实作用是让系统在出事时仍能定位责任、缩小影响、回滚变更、补充测试，并向业务、审计和监管说明发生了什么。文档只是载体，责任链和证据链才是治理的硬价值。

5.6 Agent Bricks: Databricks 对五支柱的产品化封装

Agent Bricks 是 Databricks 对生产级智能体能力的产品化封装。图 13 展示了一个 vendor-specific implementation context: 上层有 Agent Bricks、AI/BI、Custom Apps; 中间是 Agent Platform; 下层把 contextual reasoning agents、developer platform 和 AI governance 放在同一张能力地图里。

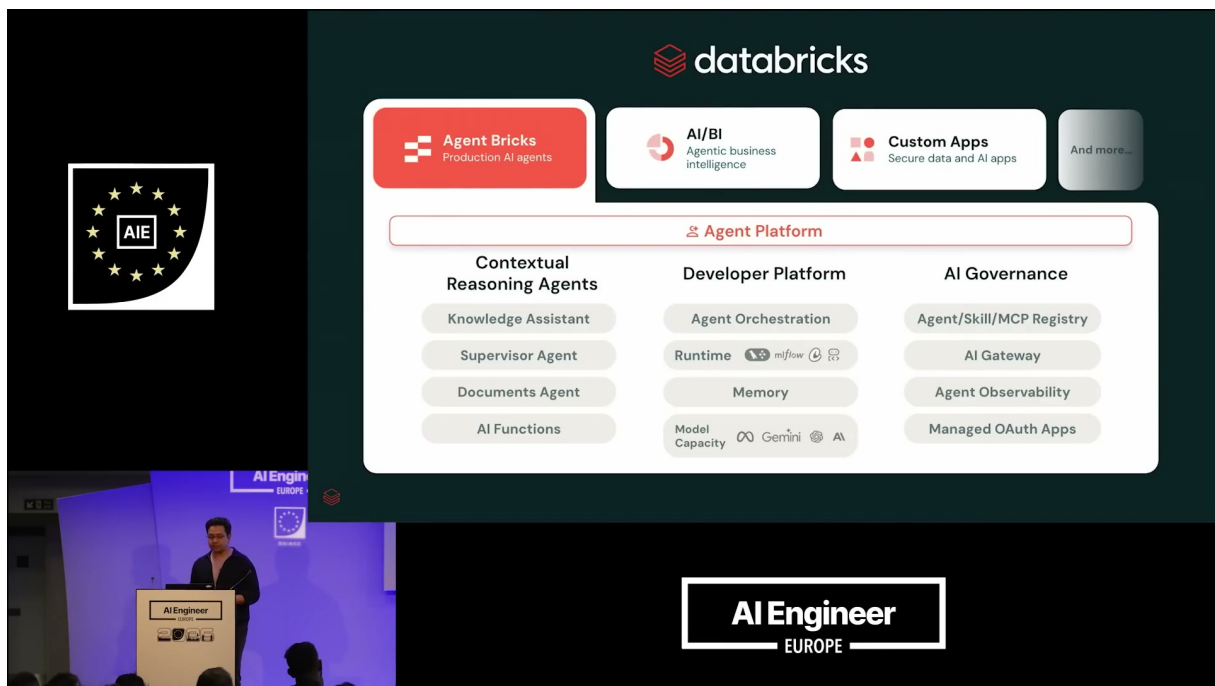


图 13: Agent Bricks 能力地图 (Agent Bricks capability map): 把 agent platform、agent governance、observability、registry、gateway 和部署能力组合成产品化路径。

12

这张图的教学价值在于把五支柱落到产品能力上: agent development platform 对应开发与部署, agent orchestration 对应多智能体协作, agent observability 对应 trace 和监控, AI gateway、managed OAuth apps、Agent/Skill/MCP registry 对应访问控制、身份和治理, MLflow 等能力承接 evaluation 与 trace-based judge。读者应把它理解为一种实现路线; 五支柱也可以由其他平台和自研组件组合实现。

从产品图抽象出通用原则

不同企业可以使用不同平台实现同一套控制面。关键在于这些能力是否真的存在: 评估数据是否可运行, trace 是否可回放, 数据权限是否统一, 智能体协作是否有状态, prompt 和模型变更是否可审计。

¹² 视频时间: 00:24:00-00:24:55, 依据 YouTube 自动英文字幕对齐。

5.7 本章小结

本章把生产级 AI 的第四、第五支柱压缩成一个控制面问题。Orchestrator-Worker 用中央控制换取可追责日志，适合复杂且需要审计的流程；Choreography 用事件总线换取并行自治，适合独立子任务和低延迟要求；Human in the Loop 用阈值和人工接管控制高风险边界。三者没有绝对优劣，关键取决于业务的依赖结构、风险等级、延迟要求和审计压力。

Governance 则回答更底层的问题：系统如何被信任。审计轨迹让组织能解释决策，PII 预验证把明确风险挡在模型之前，Prompt 版本化把自然语言配置纳入变更管理，模型变更管理让供应商升级和 fallback 选择接受企业自己的评估数据检验。到这里，五支柱已经不再是概念列表，而是一套可运行的生产控制系统。

6 Case Study：零售银行聊天机器人如何重新排序

6.1 原始失败：六个月 POC、约 85K 成本、生产失败原因不可见

讲者把前面五支柱落到一个零售银行聊天机器人案例上。这个案例的价值在于它很普通：银行希望让 AI 智能体处理简单客户问题，例如账户余额、透支费用、利率政策和常见业务咨询，从而减少人工坐席负担。也正因为普通，它能暴露许多企业 AI 项目的真实失败方式：模型先行、指标缺失、trace 缺失、责任缺失，最后 POC 花钱完成，生产系统仍然解释不了失败。

口头说明里，讲者说该银行每月大约有 20,000 次 chatbot calls，其中约 60% 属于简单查询；case-study slide 则标出 18,000 calls/month 与 60% simple queries。两者应理解为同一案例的近似口径差异：正文保留“约两万”这个口头量级，同时把 slide 上的 18,000 标为案例 slide 证据。该团队此前花了约 \$85,000 和六个月做 POC，结果失败。失败原因被 slide 总结为：system unmeasurable, invisible, misaligned。

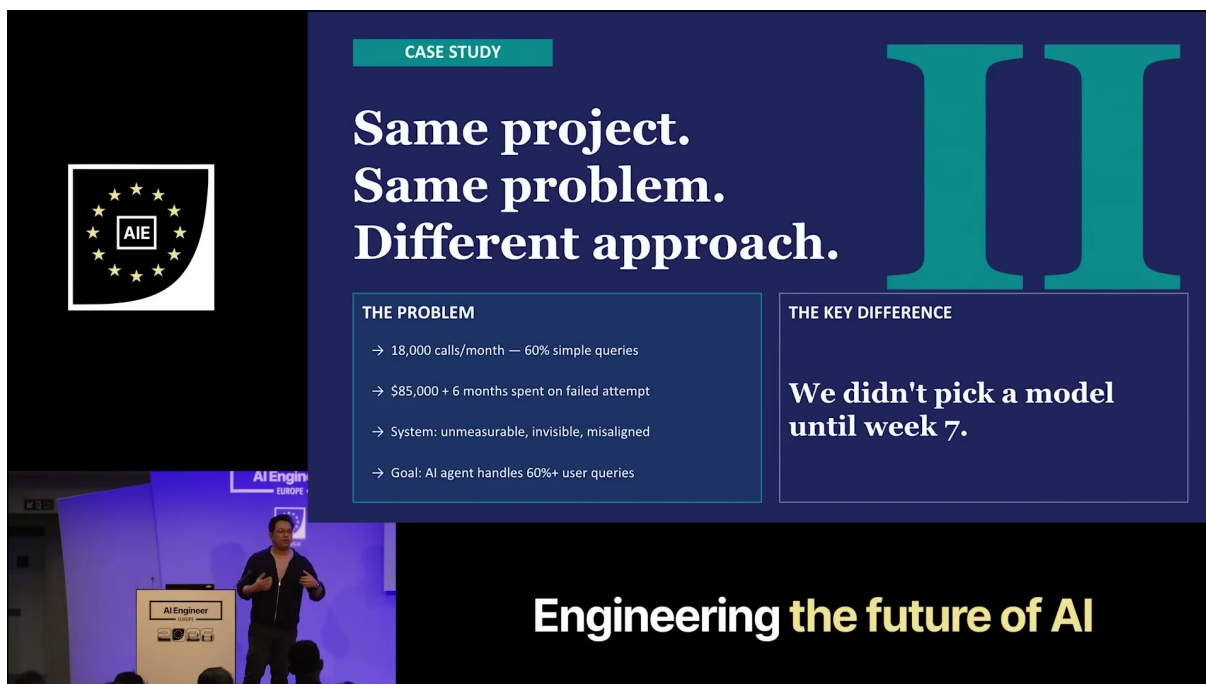


图 14: 零售银行 chatbot 案例 (Same project. Same problem. Different approach.): slide 证据显示 18,000 calls/month、60% simple queries、\$85,000 和六个月失败尝试, 并把关键差异归结为第七周才选模型。

13

这个失败的根因比“模型能力不足”更深。更准确地说, 系统没有先定义成功, 无法测量 AI 是否真的分流了简单查询; 系统缺少 trace, 无法解释用户投诉或满意度下滑; 系统缺少 accountability, 无法判断数据、prompt、工具调用、模型和人工流程到底谁该修。讲者团队重新介入后, 把目标写成更具体的业务目标: AI agent handles 60%+ user queries, 并且要能识别、追踪和验证这些被自动承接的查询。

案例的核心反转

这个银行案例的关键改变是构建顺序。团队先做 Evaluation Layer, 再做 Foundation Layer, 最后才选 AI Layer 的模型。模型选择被推迟到第七周, 原因很硬: 没有评估集和数据地基时, 模型比较只是观点争论; 有了真实测试集和 trace 之后, 模型比较变成一天内能完成的工程测量。

分流率 (Deflection Rate) 可以用一个简单公式表达:

$$D = \frac{N_{\text{agent-resolved}}}{N_{\text{eligible}}}$$

- D : 分流率, 即 AI 自动承接简单查询的比例。
- $N_{\text{agent-resolved}}$: AI 在无需人工介入时解决的简单查询数量。
- N_{eligible} : 适合 AI 承接的简单查询数量, 例如余额、透支费用、常见政策问题。

¹³ 视频时间: 00:24:50–00:26:05, 依据 YouTube 自动英文字幕对齐。

6.2 Weeks 1–2: 先建 Evaluation Layer

第 1 到第 2 周，团队没有选模型，而是建立 Evaluation Layer。这个顺序很反直觉，因为许多团队一上来就问 GPT、Claude、开源模型谁更强。案例里先做的事情更朴素：从真实人工坐席 call logs 中收集约 200 个匿名化测试案例，记录客户怎么问、人类坐席怎么答、灰区怎么处理、哪些问题需要升级。这就是黄金评估集（Golden Dataset）的雏形。

这 200 个案例的意义高于“有一批测试数据”。它们把业务专家和一线坐席的隐性知识变成可运行规格。比如，透支费用能否豁免，取决于账户历史、政策条款、客户状态和风险边界；客户问法可能含糊，答案也可能需要解释限制条件。若缺少这些真实案例，模型输出看起来流畅，实际可能错在业务细节。

接着团队定义数字化成功指标。案例中包含 60%+ 简单查询承接目标、约 85% 准确率目标、延迟目标，以及后续要追踪的客户满意度（CSAT）和人工升级规则。自动评估流水线会捕获用户问题和 AI 回复，与评估集中的期望答案比较，低于阈值时交给人工复核，并把失败案例纳入后续测试集。

Evaluation Layer 像生产系统的验收规程

传统软件里，验收测试会定义功能是否合格。AI 系统里，Evaluation Layer 承担类似角色：它定义什么叫好答案、哪些错误不能接受、何时升级给人、上线后如何持续测量。没有它，团队只能凭演示感觉判断系统质量。

6.3 Weeks 3–6: 再建 Foundation Layer

第 3 到第 6 周，团队建设 Foundation Layer。这里的 foundation 包含两类地基：一类是回答问题需要的数据和工具，例如 real-time account APIs、policy RAG、账户数据库访问和政策文档检索；另一类是追踪、审计和诊断需要的 tracking data，例如 distributed tracing、session storage、工具调用日志和 evaluator outputs。

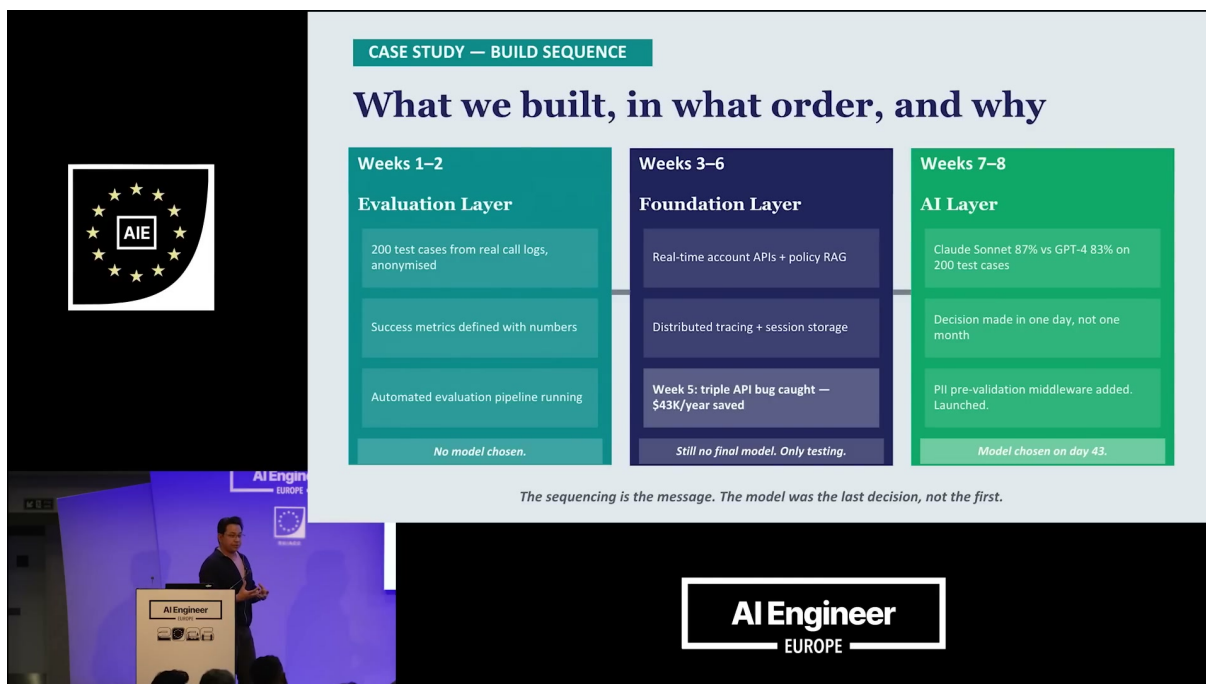


图 15: 八周 POC 的构建顺序 (Build Sequence): case-study slide 显示 Weeks 1-2 建 Evaluation Layer, Weeks 3-6 建 Foundation Layer, Weeks 7-8 才进入 AI Layer。

14

图 15 给出一个非常具体的生产收益: 第 5 周, 团队抓到了 triple API bug, slide 标注该问题修复后节省 \$43K/year。这属于 case-study slide evidence。它说明 Behavioral Evaluation 和 Observability 的价值同时体现在答案质量和工具调用成本上。用户可能拿到正确余额答案, 但如果 agent 为一次余额查询调用三次数据库, 生产成本会在高流量下被放大。

Foundation Layer 还让满意度下滑具备可诊断性。若客户给出负反馈, 团队可以沿 trace 回放: 输入是什么, 意图分类是什么, 调用了哪个账户 API, 检索到哪份政策文档, RAG 命中了哪个向量库条目, 最终回复依据是什么。没有这些数据, 团队只会看到一个模糊结果: 用户不满意。这里的未知风险在于 trace 本身可能含有敏感信息, 因此它还需要权限、脱敏、保留期限和访问审计。

数据地基的两面性

Tracking data 对调试和合规很关键, 也可能保存用户问题、账户上下文、工具返回和模型推理痕迹。企业若只收集 trace 而缺少脱敏、权限和保留策略, 会把可观测性风险转化为隐私风险。

6.4 Weeks 7-8: 最后才选模型并上线

第 7 到第 8 周, 团队才开始选模型。此时讨论已经从“谁觉得哪个模型更强”变成“哪个模型在本银行的 200 个测试案例上表现更好”。图 15 的 AI Layer 卡片给出 slide-specific numbers: Claude Sonnet 在 200 个测试案例上达到 87%, GPT-4 为 83%。这是该案例 slide 上的比较结果, 适用边界是这家银行的测试集、问题分布和评估标准。

这个顺序的效果是, 模型选择可以快速完成。slide 还写到 decision made in one day, model chosen on day 43。原因很实际: 前六周已经把评估集、自动评估流水线、真实账户 API、policy

¹⁴ 视频时间: 00:26:05-00:29:05, 依据 YouTube 自动英文字幕对齐。

RAG、distributed tracing 和 session storage 建好。模型只是最后一个可替换组件。若新模型上线前无法通过同一套评估，就延后；若表现更好且治理控制齐备，就进入发布流程。

上线前还有一个关键补丁：PII pre-validation middleware added。它对应上一章治理控制。PII 预验证运行在 LLM 之前，用 NER、regex 和规则拦截明显敏感输入或不合规请求。这样做的工程意义是降低爆炸半径：敏感内容被挡在模型调用前，日志也更容易符合审计要求。

模型被降级为可测组件

当 Evaluation Layer 和 Foundation Layer 先建好，模型选择就从战略争论降级为可重复测试。企业真正需要管理的是模型在本企业数据、工具、政策和风险阈值下的表现，公共排行榜只能作为很弱的背景信息。

6.5 Six weeks post-launch: 指标、CSAT 下滑与 stale policy 修复

上线六周后，团队开始看到生产指标。图 16 是本案例最重要的结果页，因为它既展示 ROI，也展示事故诊断能力。case-study slide 显示：准确率 87%，目标为 85%；分流率 62%，并标注 \$1M annual savings；response time 下降 65%，对应 customer wait time；CSAT 为 4.4/5，高于 humans 的 4.2。

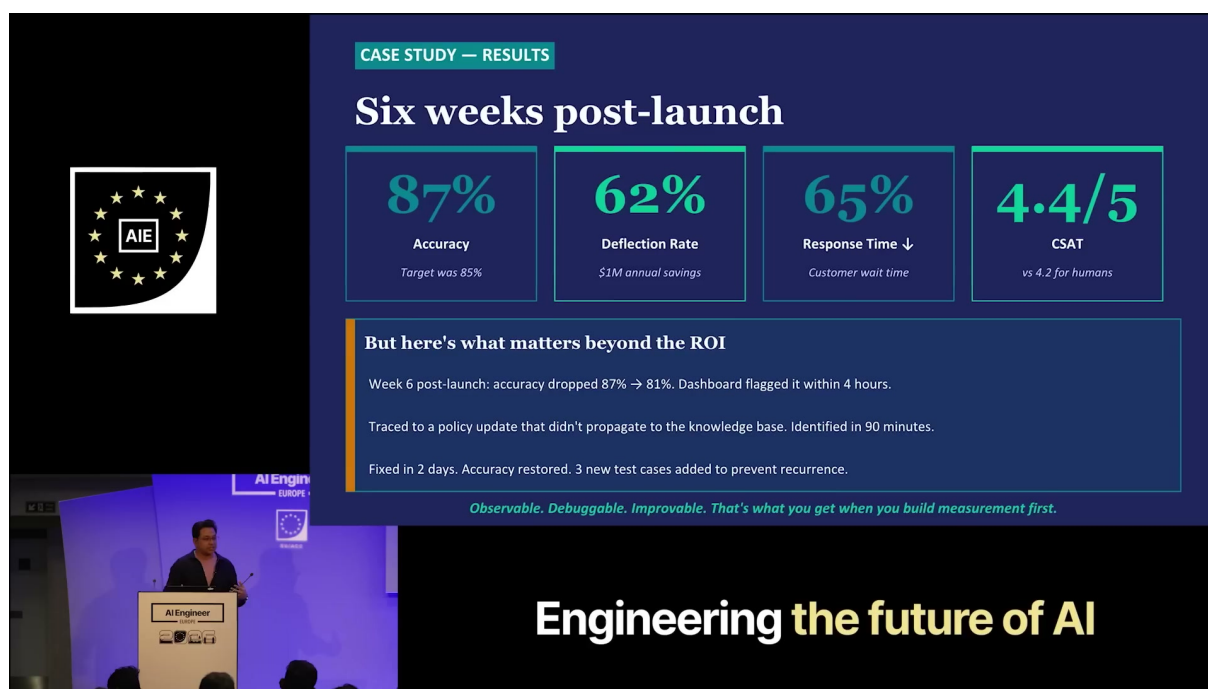


图 16: 上线六周后的案例指标 (Six weeks post-launch): slide 证据显示 87% accuracy、62% deflection rate、65% response time 下降、4.4/5 CSAT，以及一次 stale policy 事故的定位过程。

15

更有教学价值的是后半张 slide。第 6 周上线后，accuracy 从 87% 掉到 81%，dashboard 在 4 小时内标记异常。团队通过 tracing 发现根因：银行更新了利率相关政策，并通过邮件或移动银行通知客户，但新政策文档没有传播到 knowledge base；对应 embeddings 没有进入向量数据库，chatbot 继续检索旧政策，因而给出 stale answers。slide 标注该问题 90 minutes 内识别，2 days 内修复，并

¹⁵ 视频时间：00:29:10-00:30:45，依据 YouTube 自动英文字幕对齐。

新增 3 个测试案例防止复发。

这段事故说明五支柱属于动态闭环。Evaluation 发现准确率或 CSAT 下滑；Observability 回放具体失败会话；Data Foundation 揭示知识库或 embedding 更新缺口；Orchestration 决定是否转人工或降级；Governance 要求把政策更新、prompt 变更、模型变更和测试集扩展纳入流程。

stale policy 与 RAG 的直觉

RAG / vector database 像企业知识的索引目录。若政策文档已经更新，索引却仍指向旧内容，智能体就会非常自信地引用过期知识。这个问题很隐蔽，因为模型本身可能没有坏，坏的是数据更新链路和可观测闭环。

漂亮指标仍需事故诊断能力支撑

87% accuracy、62% deflection、4.4/5 CSAT 都是好信号，但真正证明系统成熟的是事故发生后的闭环速度：4 小时发现，90 分钟定位，2 天修复，3 个新测试案例固化为回归保护。

6.6 本章小结

零售银行 chatbot 案例把整场 talk 的核心压缩成一个构建顺序：先评估，再地基，最后模型。失败的六个月 POC 花了约 \$85,000，却缺少测量、trace 和 accountability；重做后的八周路径先用约 200 个匿名真实案例建立 Evaluation Layer，再用 account APIs、policy RAG、distributed tracing 和 session storage 建立 Foundation Layer，最后在企业自己的测试集上比较 Claude Sonnet 87% 与 GPT-4 83%，并在第 43 天选定模型。

上线后的数字也要按证据来源理解：fig15 作为 case-study slide evidence 显示 87% accuracy、62% deflection rate、65% response time 下降、4.4/5 CSAT、\$1M annual savings，以及一次 stale policy 事故从 87% 掉到 81%、4 小时发现、90 分钟定位、2 天修复、3 个测试案例补入评估集。案例真正想教的是生产 AI 的诊断闭环；某个模型胜出只是局部结论。每一次失败都应变成新的 trace、数据修复、测试案例、prompt 或模型变更记录。

7 Production Incident Playbook: AI 生产事故如何闭环

7.1 多数团队有部署计划，事故计划经常缺席

前一章的零售银行案例已经说明：生产级 AI (Production AI) 的真正难点，经常出现在上线之后。模型可以回答正确，工具调用路径仍然可能浪费成本；政策文档可以曾经正确，向量数据库里的版本仍然可能落后；用户反馈可以突然变差，团队仍然需要知道是数据、Prompt、模型漂移还是工具行为造成的。讲者在这一段把这些问题收束为一个工件：生产事故作战手册 (Production Incident Playbook)。

这个手册解决的是一个很现实的问题：很多团队会写部署计划，规划模型选择、上线窗口、灰度比例和发布步骤；事故发生后的检测、诊断、控制、修复和复盘流程却经常缺席。对传统软件来说，线上故障会进入告警、日志、回滚、修复和回归测试；对智能体系统来说，同样需要这条链路，只是证据变成了评估分数、链路追踪、工具调用、Prompt 版本和测试用例库。

事故计划覆盖五支柱

AI 生产事故最危险的地方在于“表面答案”和“内部行为”可以分离。用户看到的文本也许是合理答案，系统内部却可能检索了过期文档、调用了错误工具、重复访问数据库，或者绕过了应当触发人工接管的阈值。因此事故计划必须覆盖可观测性、评估、数据、编排和治理五个面向。

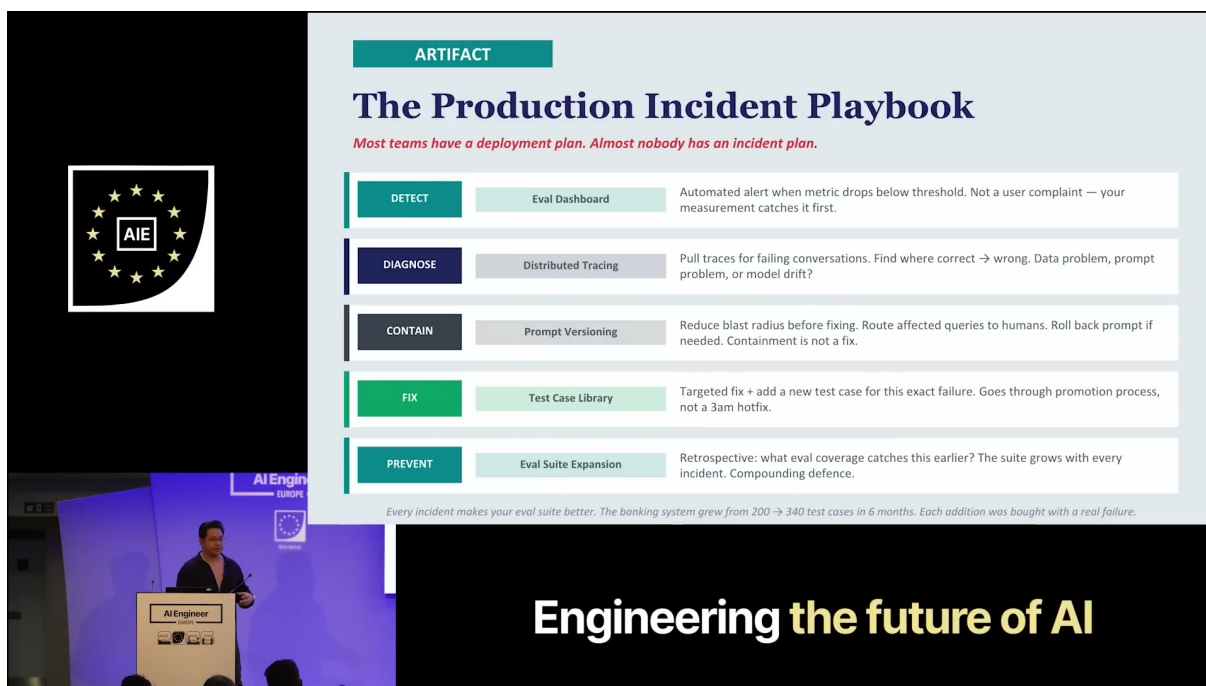


图 17: 生产事故作战手册 (The Production Incident Playbook): Detect、Diagnose、Contain、Fix、Prevent 把一次失败转化为评估套件和治理流程的增量。¹⁶

7.2 Detect: 用 Eval Dashboard 先发现指标下滑

第一步是 Detect，也就是检测。讲者强调，生产系统应当由评估看板 (Eval Dashboard) 先发现指标下滑；用户投诉通常只是更晚出现的症状。这里的“先”指工程上把测量系统放在第一响应链路中：当准确率、客户满意度 (CSAT)、分流率 (Deflection)、工具调用失败率、重复调用次数或有依据性 (Groundedness) 下降时，系统应当自动把异常暴露出来。

前一章的 stale policy 事故正好说明这一点。银行更新了利率相关政策，并向客户发出通知；聊天机器人继续从旧政策文档或旧 embedding 中检索答案，导致用户点踩、CSAT 下降。若团队只看最终投诉，定位会被拖慢；若团队已经把 eval dashboard、反馈数据和 trace 数据接起来，就能从指标下降进入追踪复盘。

¹⁶ 视频时间：00:31:40–00:32:45，依据 YouTube 自动英文字幕对齐。

Detect 要触发动作

Detect 的核心是把业务成功指标写成可触发动作的阈值，dashboard 只是承载这些阈值的界面。一个有效告警至少应回答三件事：哪个指标变坏、影响哪些请求或用户群、下一步由谁接手。

7.3 Diagnose: 用 Distributed Tracing 找到 correct 到 wrong 的转折点

第二步是 Diagnose，也就是诊断。检测只能告诉团队“哪里痛”，诊断要找出“为什么痛”。讲者给出的工具是分布式链路追踪 (Distributed Tracing)：拉出失败会话的 trace，查看智能体从正确走向错误的转折点。

这里的转折点通常落在四类根因上。第一类是数据问题，例如 RAG 的向量数据库没有收入新政策，或者 API 返回字段含义变化。第二类是 Prompt 问题，例如新版提示词改变了拒答边界、检索策略或工具选择规则。第三类是模型漂移或模型切换问题，例如供应商升级模型后，企业内部任务上的行为变了。第四类是工具行为问题，例如工具调用顺序错误、重复调用、失败重试过多、循环调用，或者应当升级给人工时仍继续自动处理。

Trace 像黑匣子

链路追踪可以理解成智能体的“黑匣子记录”。传统服务日志通常记录请求、响应、错误码和耗时；智能体 trace 还要记录中间决策：意图分类、检索到的文档、调用的工具、LLM 裁判分数、guardrail 结果和最终输出。没有这些中间证据，团队只能猜测模型为什么答错。

诊断阶段的关键产出应当是一条可被复现的失败叙事：输入是什么，系统检索了什么，调用了什么工具，哪一步从正确路径偏离，偏离后的影响是什么。这条叙事随后会进入控制、修复和预防步骤。

7.4 Contain: Prompt Versioning、转人工、降低爆炸半径

第三步是 Contain，也就是控制影响范围。讲者举的第一个控制手段是 Prompt 版本化 (Prompt Versioning)：如果某个 Prompt 版本导致了明显错误，就需要能撤回、回滚或替换它。第二个手段是把受影响查询转给人工，即人在回路 (Human in the Loop)。第三个手段是降低爆炸半径：只让高风险类别、低置信度请求或特定问题域进入保守路径，而让未受影响的请求继续服务。

讲者还提到他在另一个多智能体编排视频里讲过 Saga、补偿 (Compensation) 和熔断器 (Circuit Breaker) 等恢复模式。这里保留概念层面的定义即可：Saga 关注跨步骤任务失败后的有序恢复，补偿关注已经执行动作的反向修正，熔断器关注在下游服务或工具异常时快速停止继续放大故障。这些模式的共同目标是把事故控制在局部范围内。

Contain 只是止血

Contain 只是止血，不能替代修复。把请求转人工、回滚 Prompt 或触发熔断，可以先保护用户和业务；若团队没有把根因转化成测试用例，下一次发布仍可能重新制造同类失败。

7.5 Fix and Prevent: Test Case Library 与 Eval Suite Expansion

第四步和第五步可以合在一条反馈链里理解：Fix 是修复当前问题，Prevent 是防止同类问题再次逃逸。讲者把这条链路落在测试用例库 (Test Case Library) 和评估套件扩展 (Eval Suite Expansion)

上。

修复阶段要查看 LLM 裁判报告、评估数据集报告和失败 trace，判断修复点在哪里。可能是更新 RAG 数据源，可能是修改 Prompt，可能是修复工具调用逻辑，也可能是调整人工接管阈值。修复完成后，这个失败样本必须进入测试用例库，并配上预期答案、通过/失败标准、问题分类和责任 owner。随后评估套件扩展，让未来的发布、Prompt 变更或模型切换都能重新跑到这个案例。

失败样本变成免疫力

一次生产事故如果只带来一张故障报告，它会成为运营成本；如果它进入测试用例库和评估套件，它会成为系统免疫力。讲者在图中给出案例数字：银行系统六个月内从 200 个测试用例增长到 340 个测试用例，每一次新增都来自真实失败。

当系统接入企业既有 IT 服务管理系统（ITSM）后，事故闭环还需要明确通知、升级和责任分配。评估告警应当进入正确队列，trace 证据应当能被一线支持、平台团队和合规团队查看，修复后的测试用例应当进入发布门禁。否则，AI 事故会停留在“有人知道出问题了”的状态，无法变成可执行流程。

7.6 明天能做什么：定义成功、收集好答案、写一个最小评估流水线

讲者随后把复杂框架压缩成三件明天就能做的事。这部分很重要，因为它把前面的五支柱从架构讨论落到第一步行动。

ACTION

What to do tomorrow

Three things. In order. No exceptions.

- 01 Write your success metrics before touching a model**
Not 'accurate'. A number. A threshold. A measurement method. If you can't write this down in one sentence, you're not ready to build.
- 02 Pull 20 real examples from your use case logs**
Anonymise them. Write pass/fail criteria for each. This is your evaluation foundation. 20 is enough to start. You'll grow to 200.
- 03 Build one evaluation that runs automatically**
One deterministic check. Automated. Runs on every code change. Everything else in this talk is built on top of these three things.

If you do nothing else from today, do these three things.

AI Engineer
EUROPE

图 18: 明日行动清单 (What to do tomorrow): 先写成功指标，再抽取真实用例，最后建立一个自动运行的最小评估。¹⁷

第一，先写业务成功指标。这里的成功指标不能停在“准确率更高”这种泛化表述上，而要落成数字、阈值和测量方法。例如：简单查询的自动承接率达到某个比例，回答延迟低于某个阈值，低置信度请求必须转人工，PII 泄露命中为零，特定问题集上的正确率达到可接受标准。

¹⁷ 视频时间：00:32:45–00:33:30，依据 YouTube 自动英文字幕对齐。

第二，从真实日志中抽取 20 个用例。讲者明确说 20 个足够开始，因为最早的目标是让团队拥有真实样本、预期答案和通过/失败标准。用例需要匿名化，避免把用户个人信息带入评估数据；同时要标注类别，例如登录/安全、政策解释、账户余额、工具调用、升级人工等。

第三，写一个自动运行的最小评估流水线。这个流水线可以很朴素：输入问题，调用当前智能体，拿到回答，与预期答案或规则检查做比较，输出通过/失败和原因。即使最初只有一个确定性检查，它也已经让团队进入“每次修改都能回放同一批案例”的生产化路径。

最小 CI 类比

这三件事像软件工程里的最小 CI：先定义测试目标，再准备样本，再自动执行。区别在于 AI 系统的测试对象包括文本质量、证据来源、工具行为和治理边界，因此评估数据会随着真实故障持续增长。

7.7 本章小结

本章把生产事故作战手册拆成五步：Detect 用评估看板发现指标下滑，Diagnose 用分布式追踪找到错误转折点，Contain 通过 Prompt 版本化、人工接管和故障隔离缩小影响，Fix 修复当前根因，Prevent 把失败案例写入测试用例库并扩展评估套件。

核心结论是：生产事故不应只被视为一次线上失败，它还应成为评估数据、trace 结构、Prompt 变更记录、工具行为约束和治理流程的新增证据。AI 系统真正进入生产的标志，是每次真实失败都能被检测、解释、控制、修复，并沉淀为下一次发布前的门禁。

8 总结与延伸：三个容易漏掉的工程教训

8.1 Lesson 1: Test Case Library 需要 owner 和分类治理

讲者最后的三个教训，来自把系统推到生产之后才真正暴露的问题。第一个教训是：测试用例库 (Test Case Library) 会增长，因此它需要 owner 和分类治理。前面的银行案例从约 200 个测试用例起步，事故和修复不断把新案例补进去；图中还给出六个月后增长到 340 个测试用例的例子。增长本身是好事，因为每个新增案例都代表系统从真实失败中学习了一次。麻烦在于，没人维护的测试库很快会变成另一种技术债。

测试用例库至少需要四类元数据。第一是 owner，说明谁负责维护预期答案、分类和有效性。第二是问题类别，例如登录/安全、政策解释、工具调用、账户查询、人工升级。第三是来源，说明该案例来自生产事故、专家补充、人工坐席日志还是回归测试。第四是过期机制，说明政策变化、产品变化或数据结构变化后，哪些用例必须重新审核。

测试库也会成为技术债

测试用例库的可靠性来自治理质量，规模本身无法保证可靠。若没有 owner、分类和审核机制，旧用例可能在政策变化后 silently wrong，也就是表面仍在测试，实际预期答案已经过期。这样的测试会给团队制造虚假的安全感。

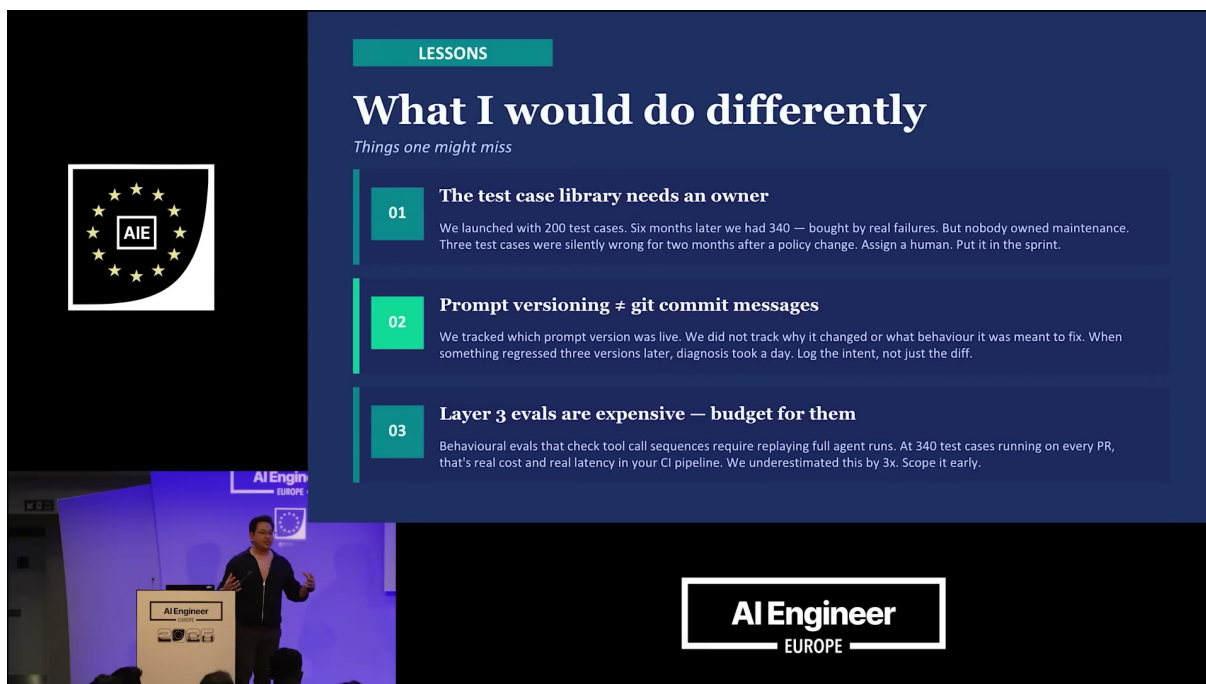


图 19: 讲者复盘的三个遗漏点 (What I would do differently): 测试用例库 owner、Prompt 版本治理、Layer 3 行为评估成本。¹⁸

8.2 Lesson 2: Prompt Versioning 的 commit message 必须能解释原因

第二个教训是 Prompt 版本化 (Prompt Versioning) 的提交信息必须解释原因。普通代码仓库里, 简短 commit message 已经经常造成追溯困难; 在 AI 系统里, Prompt 既影响语气, 也影响检索策略、拒答边界、工具选择、升级人工阈值和安全行为。一个 Prompt 变更如果只记录 “update prompt”, 事故发生后三个版本再回头看, 团队很难判断当初到底修了什么。

有效的 Prompt 变更记录应当回答四个问题: 触发这次变更的失败是什么, 期望修正的行为是什么, 影响范围是什么, 回归测试或评估案例是什么。若变更来自生产事故, 还应关联 trace、测试用例 ID、评估报告和审批人。这样 Prompt 才会进入企业变更管理, 成为带有责任链的生产资产。

Prompt 变更要记录意图

Prompt commit message 的核心职责是解释意图。它应记录 “为什么改” “修哪类失败” “如何验证”。差异文本只能告诉团队改了哪些字, 意图记录才能告诉团队这次修改承担什么生产责任。

这里还有一个容易漏掉的风险: Prompt 的行为含义会被模型版本、工具 schema 和检索数据共同改变。同一句提示词, 在不同模型或不同数据上下文里可能表现不同。因此 Prompt 版本记录最好与模型版本、工具版本和数据快照建立关联, 至少要能在事故诊断时恢复当时的执行环境。

8.3 Lesson 3: Layer 3 behavioral evals 会很贵, 需要 CI 策略

第三个教训是第三层行为评估 (Layer 3 behavioral evals) 会很贵, 需要持续集成 (CI) 策略。行为评估检查的是 agent 怎么做事: 是否调用正确工具, 是否重复调用数据库, 是否进入循环, 是

¹⁸ 视频时间: 00:33:30–00:35:40, 依据 YouTube 自动英文字幕对齐。

否在低置信度时升级人工，是否遵守工具顺序和范围边界。它比只看最终答案更接近生产真实成本，也更容易消耗预算。

成本可以用一个近似公式表达。这个公式想表达的是：总评估成本随着案例数、重复运行次数和单次运行成本线性增长；当测试用例库从几十行增长到几百行，且每一行都要完整重放 agent 运行时，PR 上的全量行为评估会迅速变成真实开销。

$$C_{\text{eval}} \approx N_{\text{cases}} \times R_{\text{runs}} \times C_{\text{per run}}$$

- C_{eval} ：一次评估批次的近似总成本，包括模型调用、工具调用、检索、judge 调用和运行时间成本。
- N_{cases} ：参与本次评估的测试用例数量。
- R_{runs} ：每个用例需要重复运行的次数；多次运行常用于观察非确定性、重试行为或 judge 方差。
- $C_{\text{per run}}$ ：单个用例单次完整运行的平均成本。

行为评估要分层运行

行为评估的成本问题不能等到测试库变大后再处理。若每个 PR 都跑 300、400、500 行完整 agent 流程，团队会在金钱成本、等待时间和开发体验之间付出代价。更稳妥的做法是在 CI 中跑小样本和高风险子集，在主分支合并或发布门禁时跑全量套件。

因此，一个可执行策略是把 eval suite 分层。PR 阶段运行确定性检查、少量语义样本和与本次改动相关的行为样本；main 分支合并运行更大的代表性集合；发布前或夜间任务运行全量行为评估。这样既保留了早反馈，也控制了日常开发成本。

8.4 资源与结束语：模板、Tracing 指南、LinkedIn/newsletter

讲者在结尾提到 QR code 和 Google Drive 资源，其中有模板、评估 checklist 示例，以及用开源技术搭建 tracing 的指南。这部分的教学价值在于提醒团队先把最小工件做出来：成功指标模板、测试用例表、评估 checklist、trace schema 和事故 playbook。工具可以逐步替换，工件结构应当尽早建立。

资源页、LinkedIn 和 newsletter 属于讲者后续交流渠道。正文保留其有用部分即可：这些材料可以帮助团队在测试环境里快速搭建 tracing，并在工具选型之前验证评估和观测流程。例行感谢、掌声和社交媒体引导不影响本讲核心机制，集成时可以从 PDF 正文略去。

模板的真实价值

模板的真实价值在于强迫团队回答关键问题：成功如何度量，失败如何分类，trace 存什么字段，谁审批 Prompt 变更，哪些测试在 PR 跑，哪些测试在发布前跑。格式漂亮只影响阅读体验；这些答案才决定生产流程能否站住。

8.5 全篇综合：从五支柱压缩成一个生产闭环

整场演讲可以压缩成一个生产闭环：评估（Evaluation）定义成功，可观测性（Observability）记录过程，数据地基（Data Foundation）供应答案与证据，编排（Orchestration）协调智能体和人工，治理（Governance）提供责任、审计、变更控制和应急响应。这个闭环把第一章的失败模式接

回最后两章的事故学习机制：demo 失败往往源于看不见、量不准、管不住；生产闭环则要求每次失败都能进入测量、追踪、修复和预防。

五支柱是反馈控制系统

五支柱共同构成一条反馈控制系统。评估给出目标函数，可观测性给出状态观测，数据地基给出可用数据和追踪数据，编排决定动作如何执行，治理决定谁负责、谁审批、谁能证明系统处在受控状态。

用一个更工程化的顺序看，闭环可以写成七步。第一，业务先定义成功指标，例如分流率、准确率、CSAT、响应延迟和 PII 违规数。第二，从真实日志和专家知识中建立黄金评估集（Golden Dataset）。第三，建立自动评估流水线，把确定性、语义和行为评估分层执行。第四，上线后收集 trace、反馈、judge 分数和工具调用记录。第五，当指标下降时，通过分布式追踪定位 data、Prompt、model drift 或 tool behavior 的根因。第六，控制影响范围并修复系统。第七，把失败样本写回测试用例库，扩展 eval suite，并更新治理记录。

这条闭环也解释了为什么模型选择应当后置。模型只是闭环中的一个可替换执行部件；真正决定生产能力的是组织能否持续回答这些问题：什么算成功，失败能否被发现，决策能否被回放，数据能否被治理，多智能体和人工如何协作，Prompt 和模型变更能否被审计。若这些问题没有答案，再强的模型也会在生产环境里变成不可解释的黑箱。

组织责任是隐藏瓶颈

这套方法的未知风险在于组织责任。很多团队能搭出 dashboard 和 trace，却没有人维护测试用例库、审批 Prompt 变更、校准 LLM 裁判（LLM-as-a-Judge）、处理 trace 中的隐私数据，或决定发布门禁标准。生产级 AI 的瓶颈经常落在组织接口、责任分配和模型能力的共同边界上。

最终，Production AI Playbook 的核心是建立一个能学习的生产系统，模型更新只是其中一个变量。每次真实失败都应留下三类资产：一个更清晰的 trace，一个更可靠的测试用例，一个更明确的治理决策。只有当这些资产持续累积，企业级智能体才会从一次漂亮演示，变成可运营、可审计、可演进的系统。

8.6 本章小结

本章补上三个容易被忽视的工程教训：测试用例库要有 owner 和分类治理，Prompt 版本记录要解释变更原因，第三层行为评估要提前设计 CI 成本策略。随后全篇被压缩为一个生产闭环：评估定义可测成功，可观测性记录决策历史，数据地基支撑答案和追踪，编排协调智能体与人工，治理给出责任、审计、变更控制和事故响应。

这场演讲的最终结论很直接：生产级 AI 的成熟度，不取决于 demo 多顺滑，而取决于系统能否把每一次真实失败转化为评估、追踪、数据、Prompt 和治理的改进。能做到这一点，AI 系统才有资格承接真实用户、真实业务和真实监管压力。